TOPIC 5

RTL Simulation and Verification

Functional verification techniques for RTL designs · Introduction to test benches and testbench development · Simulation and debugging of RTL designs

- 5a · Verification techniques — why, how much, and how you know when you are finished
- 5b · Testbenches — structure, reference models, randomisation, coverage, assertions
- 5c · Simulation and debugging — the event engine, waveforms, and a debugging procedure
- Tools · Vivado · ModelSim · Icarus + Verilator + GTKWave · Labs V1–V6

TOPIC 5 · ROADMAP

What This Topic Covers, and Why It Is Bigger Than It Looks

Topic 4 taught you to write RTL. Topic 5 asks the harder question: how do you KNOW it works? The syllabus gives this subtopic 6 theory hours; in industry it is where the majority of the engineering effort goes, and the practical component reflects that.

Syllabus bullet — Topic 5	Covered by	Slides
Functional verification techniques for RTL designs	**5a**	5–14
Introduction to test benches and testbench development	**5b**	15–46
Simulation and debugging of RTL designs	**5c**	47–56
Practical: validate RTL designs through simulation using test benches and debugging techniques	**Labs V1–V6**	62–69
Practical: develop and execute test benches for comprehensive simulation and debugging	**Tools + Labs**	57–69

Learning outcomes — by the end of this topic you can

· Write a self-checking testbench with a reference model, and say why each check is there.
· Use constrained-random stimulus with seeds, and reproduce any failure exactly.
· Define a functional coverage model and close it across a regression.
· Write assertions that catch a broken rule at the cycle it breaks.
· Debug an RTL failure by procedure rather than by guesswork.

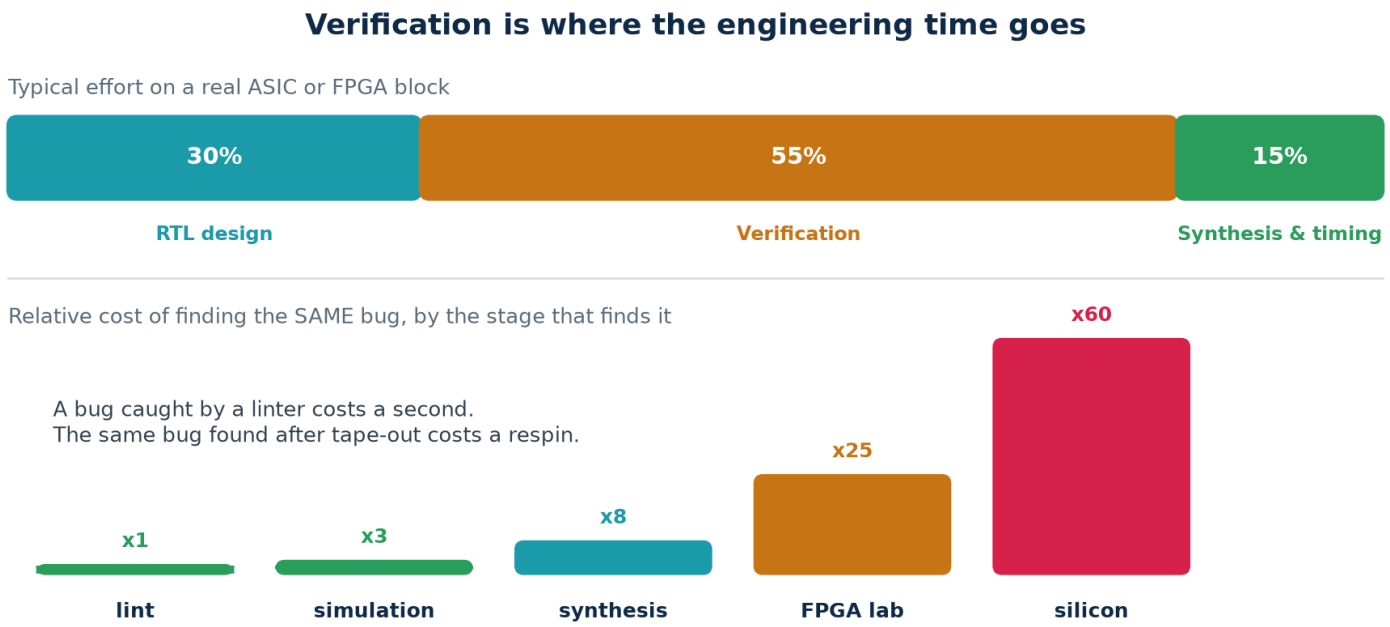
One sentence to open the session with

A testbench is not finished when it passes. It is finished when it would FAIL if the design were wrong — and this deck proves that with measured results.

TOPIC 5 · MOTIVATION

Why This Is Half the Job

Verification is not a phase that follows design. It is a separate discipline with its own tools, its own metrics and, on most teams, its own engineers — and it consumes more effort than the design itself.



Why the cost curve is so steep

A bug is the same bug at every stage. What changes is what it costs to find and fix it. Lint reports it in one second with a file and a line number. The FPGA lab makes you re-synthesise, re-place and re-programme. Silicon makes you re-spin the mask set — months, and a great deal of money — and by then your customers have the part.

TOPIC 5 · THE PROBLEM

You Cannot Test Everything — So What Do You Do?

Exhaustive testing is impossible for anything real. A single 32-bit adder has 2^64 input pairs; at a billion per second that is over 500 000 years. The state space of a design with 100 flip-flops is larger than the number of atoms in the observable universe.

The verification gap — and what closes it

The problem

- **Design size** — doubles every ~2 years
- **States to check** — grows EXPONENTIALLY with registers
- **A 32-bit adder** — 2^64 input pairs — 500 000 years
- **Exhaustive testing** — impossible for anything real

What we do instead

- **Directed tests** — for the cases you can name
- **Constrained random** — for the ones you cannot
- **Assertions** — check rules on EVERY cycle
- **Coverage** — measures what was actually reached

Not proof — evidence, measured.

So verification is not proof — it is measured evidence

You choose the cases most likely to break it (directed), let a machine choose the ones you would not have thought of (random), state the rules so they are checked continuously (assertions), and then **measure what you actually reached** (coverage). The last of those is what turns 'we tested it' into a number somebody can review.

SUBTOPIC 5A

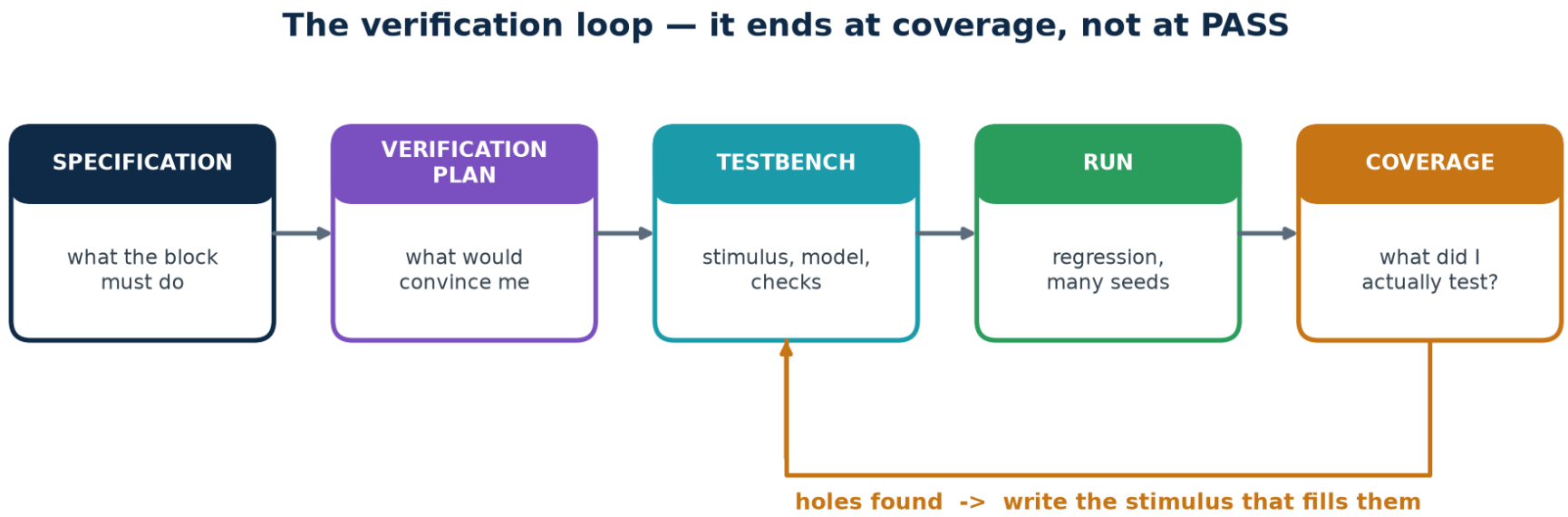
Functional Verification Techniques for RTL Designs

The methods, the metrics, and the argument for each one.

- The verification loop, and why it ends at coverage rather than at PASS
- Directed, constrained-random, and where each one earns its place
- Four ways to decide whether an answer was right
- Code coverage and functional coverage — only one of them knows the spec
- The verification plan, regression, and closure
- Measured proof: three testbenches against five broken designs

The Verification Loop

Notice where this loop ENDS. Not at 'the simulation passed' — at 'coverage is closed'. A pass with unmeasured coverage tells you only that nothing broke in the cases you happened to run.



The plan comes FIRST

Write the verification plan before the testbench, from the specification, and have it reviewed exactly as the design is reviewed. If you write the testbench first you will test what you happened to build, not what was asked for.

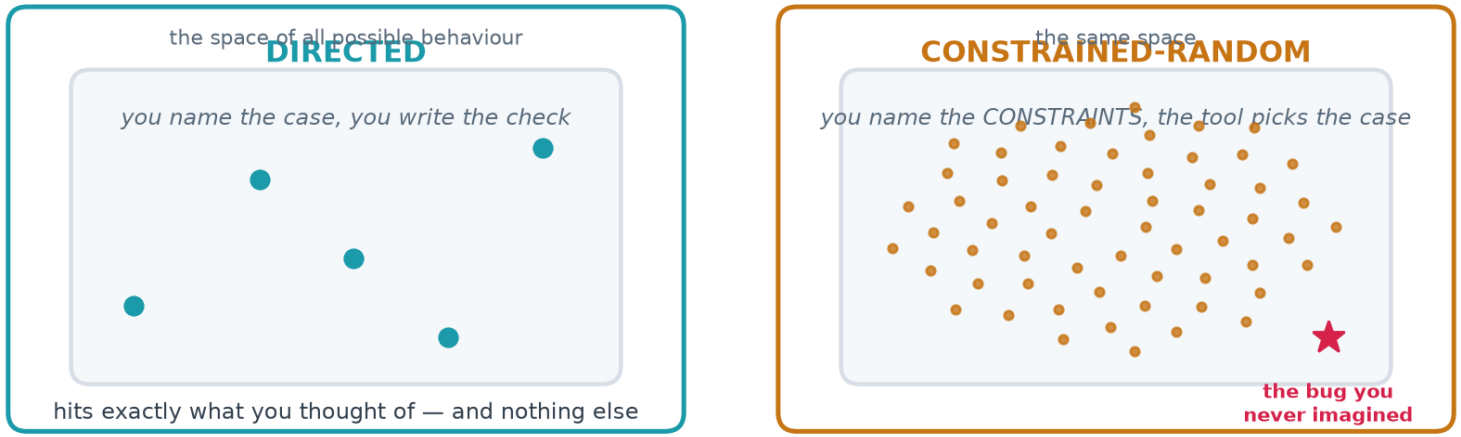
The loop closes on the holes

Coverage does not just report — it TELLS YOU WHAT TO DO NEXT. Every MISS is a specific piece of stimulus somebody has to write. That is why it is a loop and not a final report.

Directed and Constrained-Random

These are not alternatives; they are different instruments. Directed tests are precise and cheap and prove exactly what you meant. Random tests are broad and reach places you did not think of. A real project uses both, in that order.

Directed and constrained-random — you need both

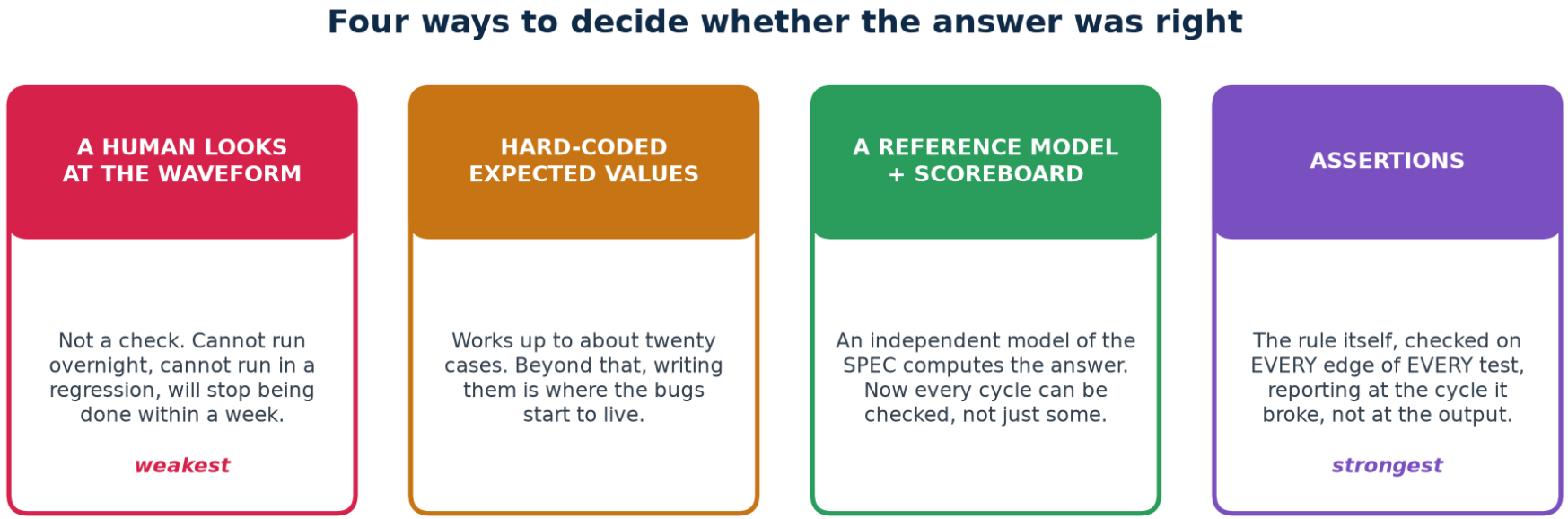


	Directed	Constrained-random
You write	the case AND the expected answer	the constraints; a model gives the answer
Cost per test	high — one test, one case	low — one test, thousands of cases
Reproducible	always	yes, from the seed
Finds	what you thought of	what you did not
Use it for	boundaries, reset, protocol corners	volume, and the long tail

TOPIC 5A · CHECKING

Four Ways to Decide Whether the Answer Was Right

This is the single biggest determinant of how good a testbench is. Not how many test cases it runs — how it decides whether each one was correct.



The line that separates a testbench from a demonstration

If deciding whether the run was correct requires a human to look at a waveform, you cannot put it in a regression, you cannot run it overnight, and you will stop running it within a week. The verdict must be a line of text a machine can grep for.

TOPIC 5A · COVERAGE

Code Coverage and Functional Coverage

Coverage answers the question a passing test cannot: WHAT DID I ACTUALLY TEST? There are two kinds, they measure different things, and you need both.

Two kinds of coverage — and only one of them knows the spec

CODE COVERAGE

computed by the tool, automatically

line	was this line executed?
branch	was each if/else arm taken?
condition	did each sub-expression take both values?
toggle	did each bit go 0->1 and 1->0?
FSM state	was each state entered, each arc taken?

100% code coverage still proves nothing about behaviour you never wrote code for.

FUNCTIONAL COVERAGE

written by YOU, from the specification

- did the FIFO ever reach full?
- read and write on the same cycle?
- ...while empty? ...while full?
- did the pointers wrap?
- full -> empty -> full?

This is the list that says when you are FINISHED.

The trap that catches every team once

100% code coverage does not mean the design is verified. It means every line you wrote was executed. It says nothing about the behaviour you FORGOT to write code for, nothing about whether the outputs were checked, and nothing about whether the FIFO ever went full. Functional coverage is the one written from the specification — and it is the one that says when you are finished.

The Verification Plan Is One Table

A verification plan does not need to be a document. It needs to be a table, written before the testbench, reviewed like the design, with one row per feature in the specification and a column that says how each is checked.

A verification plan is one table, written before the testbench

Feature from the spec	How it is checked	Stimulus	Coverage bin	Status
Words come out in order	scoreboard, every cycle	random	write/read accepted	done
Never writes when full	assertion a_full_iff_depth	write-heavy	write while full	done
Never reads when empty	assertion a_count_range	read-heavy	read while empty	done
Simultaneous r+w holds count	assertion a_step_both	balanced	r+w same cycle	done
r+w while EMPTY writes	scoreboard	balanced	r+w while empty	done
Reset empties the FIFO	directed check	directed	post-reset state	done
Pointers wrap correctly	scoreboard	long random	pointers wrapped	done

Written FIRST, reviewed like the design, and the 'Status' column is what the project manager actually asks about.

Why writing it first matters

A plan written afterwards documents what you happened to build. A plan written first tells you what to build — and the gaps are visible while they are still cheap to close.

Why the Status column matters

It is the only honest answer to "are we done?". Not "the tests pass" — **"these features are checked, by these mechanisms, and here are the ones that are not".**

TOPIC 5A · THE FILTERS

Each Stage Catches What the Previous One Cannot

No single technique catches everything. Each is a filter with a different mesh, and skipping one lets a whole class of bug through to the next — where it costs more.

Each stage is a filter — and each one you skip lets bugs through

Run them cheapest first. A bug that a one-second lint would have named should never reach a waveform viewer.

Run them in cost order, every time

Lint is one second and catches width truncation and inferred latches. Simulation is a minute and catches wrong behaviour. Assertions cost nothing extra once written and catch a broken rule at the cycle it breaks. Coverage catches what was never exercised at all. Synthesis catches the structural surprises the simulator hid from you.

TOPIC 5A · THE PROOF

Three Testbenches, Five Broken Designs, One Table

Everything in this topic reduces to this experiment. One correct FIFO and five copies with a single realistic bug each. All five lint clean, all five synthesise, and all five pass a testbench that most students would call finished.

The whole of Topic 5, in one table — measured, not asserted

The same three testbenches, run against the correct FIFO and five broken copies. Every cell is real output from Topic5_Lab.

testbench	fifo	b1	b2	b3	b4	b5	bugs caught
V1 naive directed	pass	**pass**	**pass**	**pass**	**pass**	**pass**	**0 of 5**
V2 model + corners	pass	**CAUGHT**	**CAUGHT**	**CAUGHT**	**CAUGHT**	**pass**	**4 of 5**
V3 constrained-random	pass	**CAUGHT**	**CAUGHT**	**CAUGHT**	**CAUGHT**	**CAUGHT**	**5 of 5**

"pass" on a BROKEN design (red) means the testbench MISSED the bug.

"CAUGHT" (green) means it failed — which on a broken design is the CORRECT result.

A testbench is not finished when it passes. It is finished when it would fail if the design were wrong.

Run it yourself in ten seconds

`cd Topic5_Lab && ./scripts/clinic.sh` — every cell in that table is produced by that command, on the code in the lab folder. Show it to the room before you explain anything else; the rest of the topic is the explanation of why the numbers come out that way.

TOPIC 5A · READING THE TABLE

What Actually Changed Between V1, V2 and V3

The three testbenches do not differ in size or effort. They differ in three specific decisions, and each decision is worth a row of that table.

V1 → V2	Stopped typing expected values by hand and built a REFERENCE MODEL — an independent implementation of the specification. Once you have one, you can check every cycle instead of the few you remembered. Then added the boundary cases: fill to full, drain to empty, write while full, read while empty. Result: 0 of 5 becomes 4 of 5.
V2 → V3	Replaced hand-written stimulus with WEIGHTED RANDOM stimulus, seeded so any failure reproduces exactly. The fifth bug only appears when a read and a write are asserted together on an EMPTY FIFO — a combination no directed test in V2 ever produces, and one nobody thinks to write. Result: 4 of 5 becomes 5 of 5.
V3 → V4	Added a COVERAGE MODEL, so a run that passes also reports what it reached. Without it you cannot tell a thorough run from a lucky one — and the write-heavy profile, which passes cleanly, never reaches empty at all.
V3 → V6	Added ASSERTIONS bound to the DUT, so a broken rule is reported at the cycle it breaks with the name of the rule — rather than three hundred cycles later, at an output, as a wrong number.

TOPIC 5A · CHECKPOINT

Eight Questions Before We Build Anything

Ask the room. These are all conceptual — no code yet. Answers are in workbook section T5-A.

Questions 1–4

1. Why is 100% code coverage not the same as "verified"?
2. A testbench passes on a design you know is broken. What does that tell you about the testbench?
3. Why must a reference model be written from the specification and not from the RTL?
4. What does the SEED give you that makes random stimulus usable in a regression?

Questions 5–8

5. Name one bug an assertion catches earlier than a scoreboard, and one it cannot catch at all.
6. What is the difference between a coverage HOLE and a test FAILURE?
7. Why does a regression usually dump no waveforms?
8. Your random test has passed 10 000 seeds. Are you finished? What would you need to see?

If question 2 or 8 gave the room trouble

Those two are the whole topic. Go back to the clinic table and run `./scripts/clinic.sh` live before continuing — the measured result convinces people in a way that an argument does not.

SUBTOPIC 5B

Introduction to Test Benches and Testbench Development

Built up in six stages, each one measured against five broken designs.

- The six parts every testbench has, and the DUT under test
- Clock, reset, and when to drive and when to sample
- Reference models and scoreboards
- Constrained-random stimulus, seeds and regressions
- Functional coverage, and closing it
- Assertions, and a layered testbench environment

One Design, Six Testbenches — Meet the FIFO

Everything in 5b is built against a single device under test. A small synchronous FIFO — big enough to have interesting corners, small enough to hold in your head. Its header comment IS the specification your testbench checks.

Topic5_Lab/rtl/fifo.v — the interface contract

```
//  * A write is accepted when wr_en=1 and full=0. A write while FULL is IGNORED
//    and must not corrupt anything.
//  * A read is accepted when rd_en=1 and empty=0. A read while EMPTY is IGNORED.
//  * rd_data shows the OLDEST unread word at all times (first-word fall-through).
//  * count is the number of words stored, 0..DEPTH.
//  * empty <=> count==0.   full <=> count==DEPTH.
//  * Words come out in the ORDER they went in. None is lost or duplicated.
//  * Asynchronous active-low reset empties the FIFO.
```

Every line of that comment becomes a check

There are eight bullets. A finished testbench has a check for each one, and a coverage bin proving the situation actually arose. That mapping — spec bullet to check to bin — is the whole verification plan.

And five broken copies

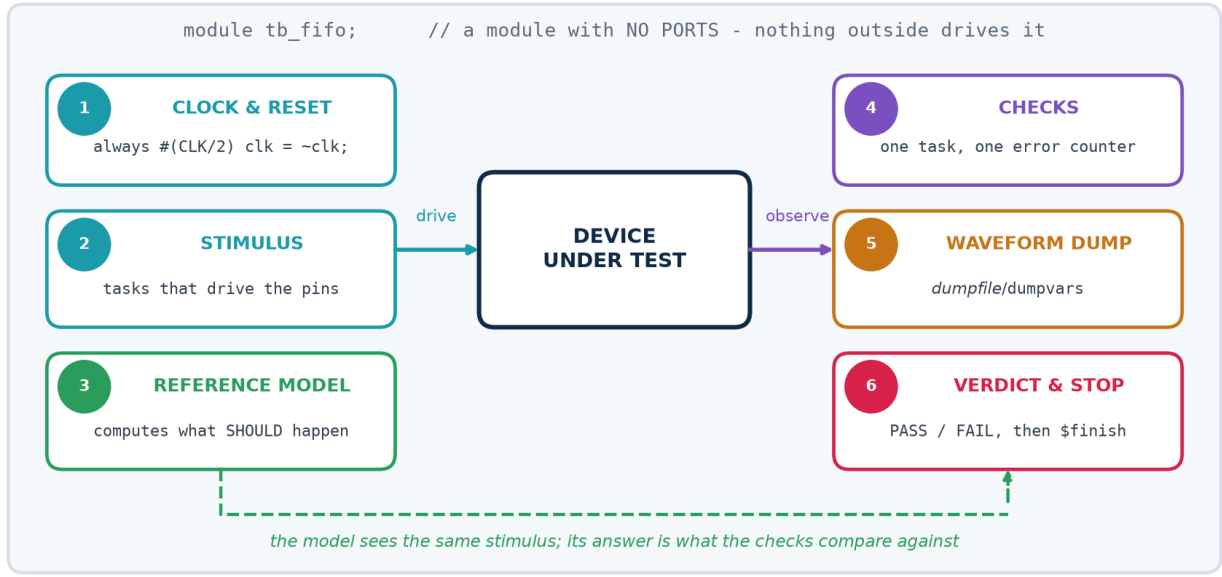
`Topic5_Lab/rtl/fifo_bugs.v` holds five variants with one realistic bug each. They all lint clean, all synthesise, and all pass a naive testbench. Do not read the bug list before attempting the clinic.

TOPIC 5B · ANATOMY

The Six Parts of Every Testbench

A testbench is a module with no ports. It is never synthesised, so the entire language is available — and every testbench you will ever write has these six parts, whatever the design.

The six parts of every testbench



Missing part 6 is the commonest omission

A testbench that never calls **`$finish`** runs until somebody kills it, which in a regression means the whole run hangs. Add a watchdog too: a second initial block that prints FAIL and finishes after a generous timeout, so a hung DUT FAILS rather than stalling the regression.

The Testbench Everybody Writes First

Topic5_Lab/tb/tb_v1_naive.v — all six parts, and it does check its results

```
module tb_v1_naive;
  fifo #(.W(8), .DEPTH(8)) u_dut (.clk(clk), .rst_n(rst_n), ...);   // 1. instantiate

  // 2. ONE place that decides pass or fail
  task check(input [7:0] got, input [7:0] exp, input [255:0] msg);
    if (got !== exp) begin
      $display("FAIL %0t : %0s  got %h expected %h", $time, msg, got, exp);
      errors = errors + 1;
    end
  endtask

  initial begin
    $dumpfile("v1.vcd");  $dumpvars(0, tb_v1_naive);          // 3. waveform dump
    rst_n = 1'b0; repeat (3) @(posedge clk); #1 rst_n = 1'b1;  // reset, released
                                                              //   BETWEEN edges
    push(8'hA1); push(8'hB2); push(8'hC3);                    // 4. stimulus
    pop(got); check(got, 8'hA1, "first word out");            //    and checks
    pop(got); check(got, 8'hB2, "second word out");
    pop(got); check(got, 8'hC3, "third word out");

    if (errors == 0) $display("PASS - 3 words in, 3 words out");  // 5. verdict
    $finish;                                                      // 6. stop
  end
endmodule
```

This is not a bad testbench. It has every structural part, and it decides pass or fail without a human. It is still worth ***nothing*** — and the next slide says why.

TOPIC 5B · LAB V1

…and It Passes on Every Broken Design

Run that testbench against the golden FIFO and against all five broken ones. This is real output.

Verified output

```
$ ./scripts/clinic.sh

  V1 naive directed      pass      pass      pass      pass      pass      pass
                         fifo      fifo_b1   fifo_b2   fifo_b3   fifo_b4   fifo_b5

  V1 : 0 of 5 bugs caught, golden design passes (no false alarm)
```

Why it misses everything

It never fills the FIFO. It never empties it past zero. It never asserts read and write on the same cycle. It never wraps the pointers. It never attempts a write while full or a read while empty.

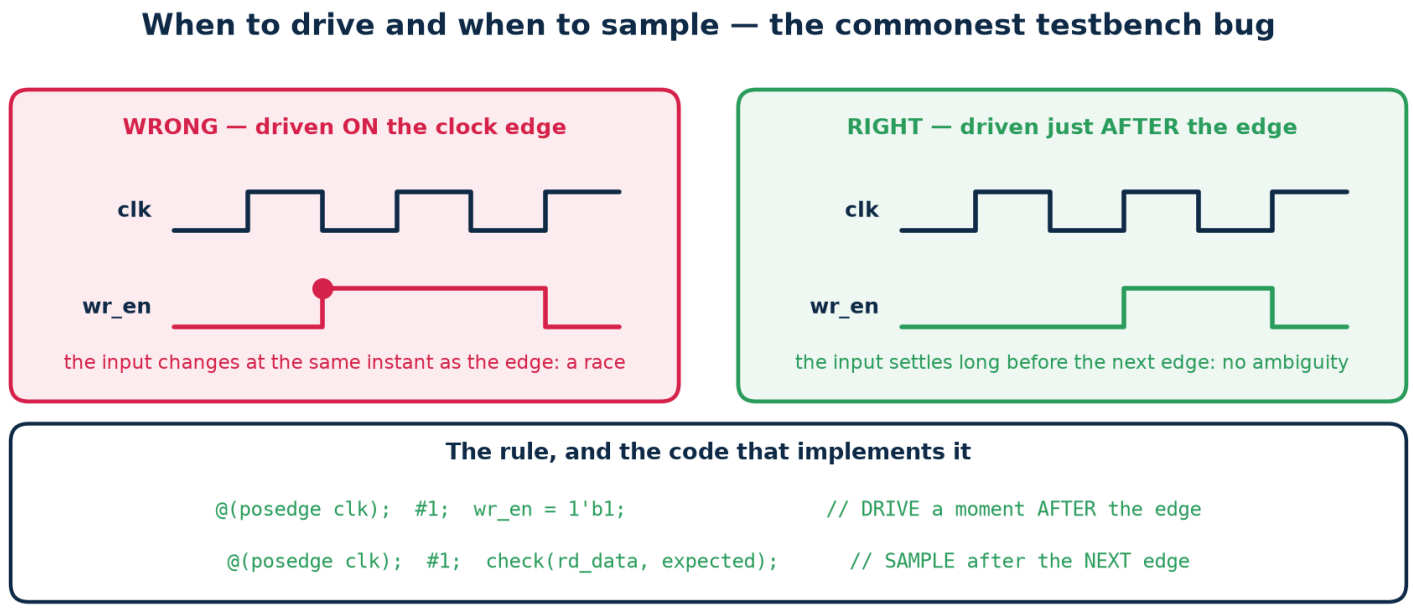
Three words in and three words out exercises the one path where a broken FIFO and a correct FIFO behave identically. The bugs are all at the boundaries — which is where bugs generally are.

The lesson to state explicitly

Coverage of the SPECIFICATION, not volume of stimulus, is what makes a testbench worth having.

When to Drive and When to Sample

More 'the design is broken' reports come from this than from any other cause. If you change an input at the same instant as the clock edge, you have created a race — and which value the DUT sees is not defined.



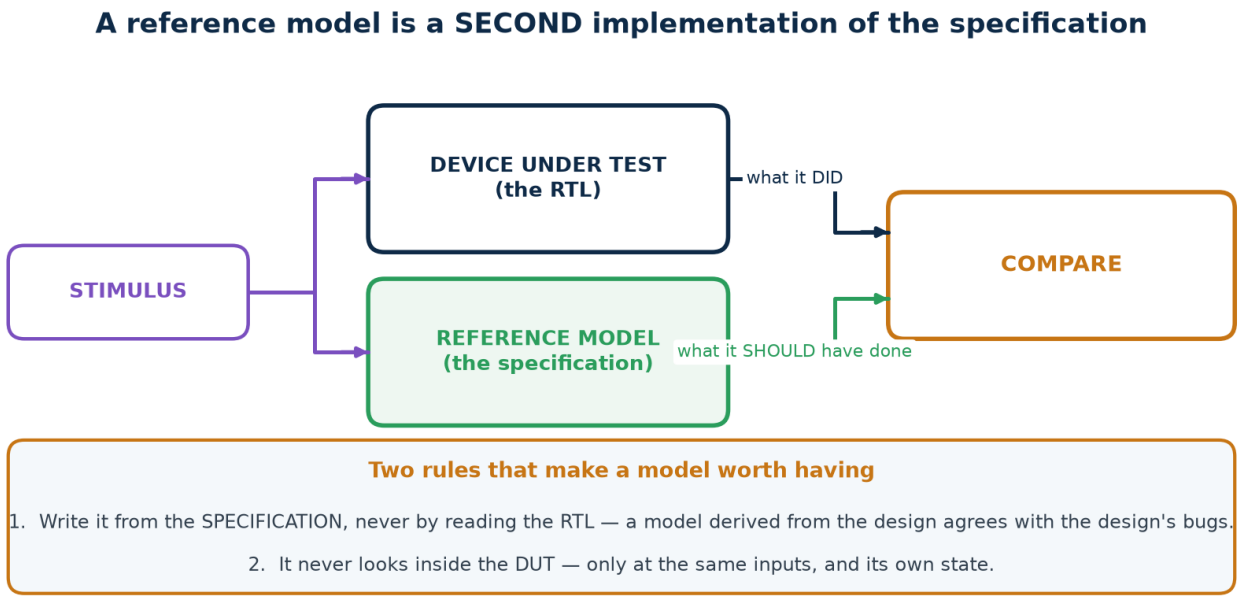
Two real examples from these labs

Topic 4: an edge-detector test failed because the stimulus changed mid-cycle, so the pulse was half a cycle wide and the check sampled at the wrong moment.

Topic 4: an FSM testbench released reset exactly ON a clock edge; the state register saw an ambiguous condition and stayed x for two cycles, and the sequence detector found one match instead of two. Neither design was broken.

The Step That Changes Everything

Hand-written expected values do not scale past about twenty cases. A reference model is a second, independent implementation of the specification — usually far simpler than the RTL, because it need not be efficient or synthesisable.



Topic5_Lab/tb/tb_v2_selfcheck.v

```
// The FIFO's model: an array and two indices. That is all it needs to be.
reg [W-1:0] model [0:DEPTH-1];
integer     mhead, mtail;            // mtail - mhead is the occupancy
function [$clog2(DEPTH):0] mcount; input dummy; mcount = mtail - mhead; endfunction
```

Once the model exists you can compare EVERY cycle — count, empty, full and the output word — instead of only at the points where you happened to write a check.

A Real Bug — in the Reference Model

The model is code, and code has bugs. This one was found while writing this lab, and it is the most instructive failure in the whole topic: the CORRECT FIFO failed, and a BROKEN one passed.

A real bug in a reference model, found while writing this lab

Read and write asserted together, on an EMPTY FIFO.

THE MODEL, WRITTEN WRONG

```
if (do_w && !mfull())  push(d);
if (do_r && !mempty()) pop();
```

start	model empty, 0 words
push	model now holds 1 word
test mempty()	FALSE — it just pushed
pop	the word goes straight back out
result	model says 0. Hardware says 1.

THE MODEL, WRITTEN RIGHT

```
was_full  = mfull();      // sample FIRST
was_empty = mempty();
if (do_w && !was_full)  push(d);
if (do_r && !was_empty) pop();
```

The hardware decides BOTH from the state that existed BEFORE the clock edge. So must the model.

Symptom before the fix: the CORRECT FIFO failed, and the BROKEN one passed.

The general rule this teaches

A model must make its decisions from the same state the hardware does. Hardware evaluates every enable from the state that existed BEFORE the clock edge, all at once. A model that applies one operation and then tests its own updated state is modelling something the hardware never does. Sample first; then apply.

Model Plus Corners — 4 of 5

V2 adds two things to V1: the reference model, and the boundary cases. It is not a longer test — it is a better-chosen one.

The eight directed tests in tb_v2_selfcheck.v

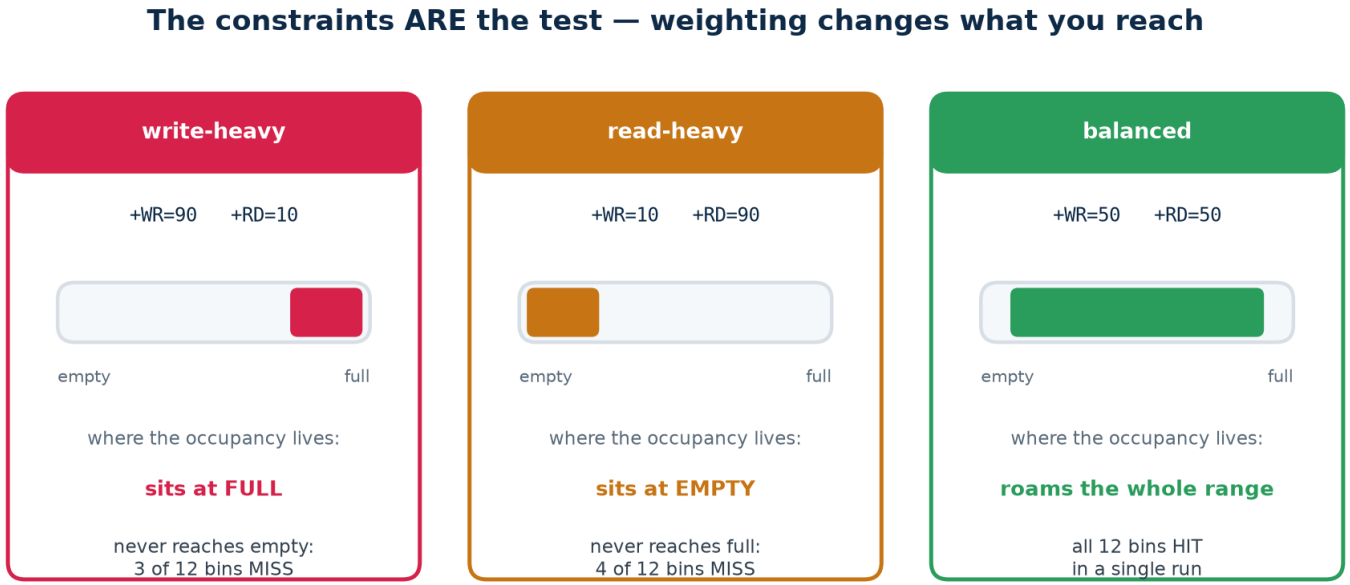
```
T1  state immediately after reset
T2  fill to exactly full            -> full must assert, empty must not
T3  a write WHILE FULL              -> must be ignored, occupancy unchanged
T4  drain to exactly empty          -> empty must assert, full must not
T5  a read WHILE EMPTY              -> must be ignored, occupancy unchanged
T6  read and write on the SAME cycle, held half full
T7  keep going long enough to WRAP the pointers
T8  final drain, and the FIFO must end empty
```

```
$ ./scripts/clinic.sh
  V2 model + corners     pass     CAUGHT   CAUGHT   CAUGHT   CAUGHT   pass
  V2 : 4 of 5 bugs caught, golden design passes (no false alarm)
```

Four bugs, from eight well-chosen directed tests and a model. Nothing exotic — just the boundaries, checked against something that knows the right answer.

The Constraints ARE the Test

Constrained-random does not mean 'random'. You still decide what the test does — you just express it as weights and let the generator produce the sequences. Change the weights and you are testing a different part of the design.



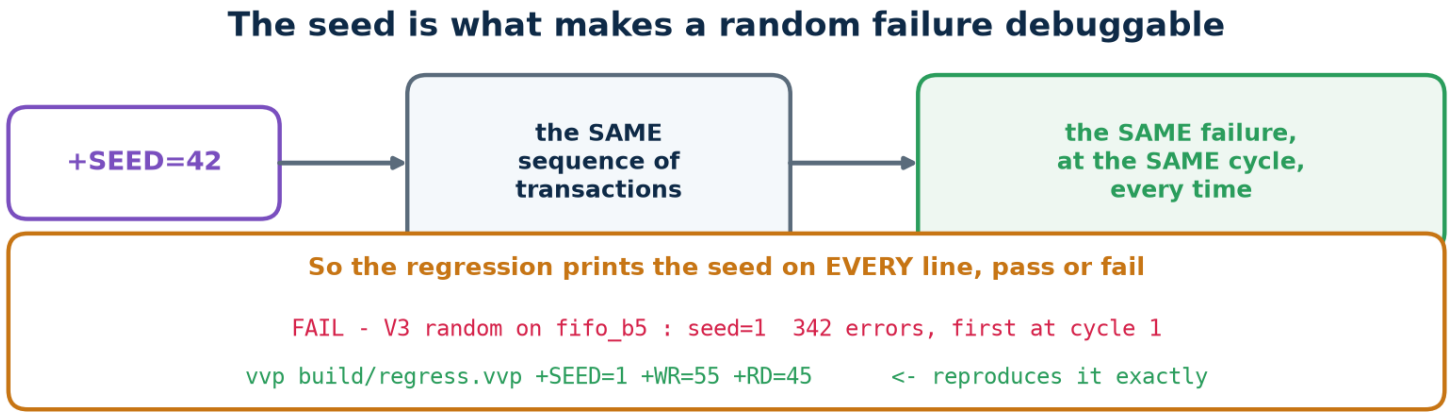
Verified in Topic5_Lab: no single profile closes coverage. Merged across all three, it does.

Topic5_Lab/tb/tb_v3_random.v — three lines of generator

```
do_w = (({$random(seed)} % 100) < p_wr);      // p_wr and p_rd are the CONSTRAINTS
do_r = (({$random(seed)} % 100) < p_rd);
d    = $random(seed);
```

Reproducibility Is What Makes Random Usable

A random failure you cannot reproduce is not a bug report — it is a rumour. The seed makes every run deterministic, so any failure can be replayed exactly, with waveforms this time.



Verified output

```
$ ./scripts/regress.sh 4        # 4 seeds x 3 profiles
  profile        seed     cycles   result
  balanced       1        3000     pass
  balanced       2        3000     pass
  write-heavy    1        3000     pass
  read-heavy     1        3000     pass
  ...
  12 passed, 0 failed
  REGRESSION CLEAN
```

Two traps: `$random(seed)` UPDATES seed in place, so save the original before the loop if you want to print it; and in SystemVerilog `$urandom(seed)` RE-SEEDS on every call — seed once, then call `$urandom()` with no argument.

Constrained-Random — 5 of 5

The full clinic result — verified

```
$ ./scripts/clinic.sh

  testbench              fifo      fifo_b1   fifo_b2   fifo_b3   fifo_b4   fifo_b5
  ------------------------------------------------------------------------------
  V1 naive directed      pass      pass      pass      pass      pass      pass
  V2 model + corners     pass      CAUGHT    CAUGHT    CAUGHT    CAUGHT    pass
  V3 constrained-random  pass      CAUGHT    CAUGHT    CAUGHT    CAUGHT    CAUGHT

  V1 : 0 of 5 bugs caught, golden design passes (no false alarm)
  V2 : 4 of 5 bugs caught, golden design passes (no false alarm)
  V3 : 5 of 5 bugs caught, golden design passes (no false alarm)
```

What the fifth bug was, and why only random found it

`fifo_b5` drops a write when wr_en and rd_en are asserted together while the FIFO is EMPTY. That is a legal, sensible combination — and no directed test in V2 ever produces it, because nobody thinks to write "read and write at once, from an empty FIFO" as a test case.

The random generator produced it ten times in three thousand cycles without being asked. That is the entire argument for constrained-random stimulus.

And note the first column

Every testbench passes on the golden design. A checker that fires on correct hardware is worse than no checker — the team learns to ignore it.

Writing a Coverage Model by Hand

SystemVerilog has covergroups. Plain Verilog does not — so you write one counter per interesting situation. Doing it by hand once is the best way to understand what the tool does for you later.

Topic5_Lab/tb/tb_v4_coverage.v

```
// THE COVERAGE MODEL. Write this list BEFORE the stimulus.
//   0  write accepted            6  occupancy reached DEPTH (full)
//   1  read accepted             7  occupancy reached 0 (empty)
//   2  read+write same cycle     8  pointers wrapped at least once
//   3  idle cycle                9  write attempted while full
//   4  read+write while EMPTY   10  read attempted while empty
//   5  read+write while FULL    11  a full -> empty -> full round trip

if (do_w && !was_full)         cov[0] = cov[0] + 1;      // sample, every cycle
if (do_w && do_r && was_empty) cov[4] = cov[4] + 1;
```

Bin 11 is the one worth pointing at

"full → empty → full" is not a state; it is a SEQUENCE over time, and it needs a little state machine in the testbench to detect. Real coverage models are full of these — 'a request followed by a retry', 'back-to-back bursts', 'reset in the middle of a transfer'. They are also where the interesting bugs live.

Coverage Holes, and Closing Them

Here is the result that surprises students: a run can PASS and still prove almost nothing. The write-heavy profile passes cleanly — and never reaches empty at all.

A coverage model, and the report it produces

Twelve bins, written before the stimulus. Real output from Topic5_Lab, three stimulus profiles.

coverage bin	write-heavy	read-heavy	balanced	MERGED
write accepted	HIT	HIT	HIT	HIT
read accepted	HIT	HIT	HIT	HIT
read+write same cycle	HIT	HIT	HIT	HIT
idle cycle	HIT	HIT	HIT	HIT
read+write while EMPTY	MISS	HIT	HIT	HIT
read+write while FULL	HIT	MISS	HIT	HIT
reached FULL	HIT	MISS	HIT	HIT
reached EMPTY	MISS	HIT	HIT	HIT
pointers wrapped	HIT	HIT	HIT	HIT
write attempted while full	HIT	MISS	HIT	HIT
read attempted while empty	MISS	HIT	HIT	HIT
full -> empty -> full	MISS	MISS	HIT	HIT

No single profile closes coverage. Merged across the regression, it does — and that is how coverage is really closed.

This is what closure looks like in practice

No single profile closes coverage, and none has to. Each run contributes what it reached, the results are MERGED across the regression, and what is still missing is the specification for the next test you write. `./scripts/coverage.sh` does exactly this and prints COVERAGE CLOSED.

Stating the Rule Instead of Checking the Output

A scoreboard checks the outputs at the points you compare them. An assertion states a RULE, and the simulator checks it on every clock edge of every test, for ever — and reports at the exact cycle and the exact line where it broke.

Anatomy of a concurrent assertion

```
a_step_up: assert property (@(posedge clk) disable iff (!rst_n)  wr_accepted |=> count == $past(count)+1);
```

`a_step_up:`	a NAME — it appears in the failure message
`assert property`	check this on every clock. cover property just RECORDS it.
`@(posedge clk)`	the sampling clock — assertions are synchronous
`disable iff (!rst_n)`	switched off during reset, where nothing is guaranteed
`|=>`	implication with ONE cycle of delay. |-> means same cycle.
`$past(count)`	the value one clock ago — how you express 'changed by'

assert versus cover

`assert property` — this must ALWAYS be true; fail loudly if not.

`cover property` — this is not a rule, just RECORD whether it ever happened. That is functional coverage, expressed in the same language.

Where to put them

In a separate module bound to the DUT's signals — as in **`sva/fifo_sva.sv`**. The design source stays clean, the assertions can be maintained by the verification engineer, and they can be switched off for synthesis without touching the RTL.

The FIFO's Specification, Written as Properties

Topic5_Lab/sva/fifo_sva.sv — lint-clean under Verilator 5.020

```
// 1. count and the flags must always agree
a_empty_iff_zero: assert property (@(posedge clk) disable iff (!rst_n)
    empty == (count == 0))       else $error("empty=%0b but count=%0d", empty, count);

a_full_iff_depth: assert property (@(posedge clk) disable iff (!rst_n)
    full == (count == DEPTH_C))  else $error("full=%0b but count=%0d", full, count);

// 2. occupancy may only move by one per cycle, in the right direction
a_step_up:   assert property (@(posedge clk) disable iff (!rst_n)
    (wr_en && !full && !(rd_en && !empty)) |=> (count == $past(count) + 1));

a_step_both: assert property (@(posedge clk) disable iff (!rst_n)
    (wr_en && !full && rd_en && !empty)    |=> (count == $past(count)));

// 3. COVER: not checks. They record that the interesting situations HAPPENED.
c_both_empty: cover property (@(posedge clk) disable iff (!rst_n)
    wr_en && rd_en && empty);
```

Read the property names aloud and they are the specification: "empty if and only if count is zero", "a write steps the count up by one". That readability is the point — an assertion nobody can read is an assertion nobody will maintain.

TOPIC 5B · LAB V6

Which Caught Which — and the One That Slipped Through

Assertions and scoreboards are not alternatives. Here is the measured proof, from the same five broken FIFOs.

Assertions and scoreboards catch different things

Real output from ./scripts/assert.sh in Topic5_Lab.

broken design	what is wrong	caught by	when
`fifo_b1`	full flag is one entry late	ASSERTION a_full_iff_depth	205 ns
`fifo_b2`	read not guarded when empty	ASSERTION a_count_range	425 ns
`fifo_b3`	count drifts on simultaneous r+w	ASSERTION a_step_both	545 ns
`fifo_b4`	write address wrong after a wrap	SCOREBOARD only	456 ns
`fifo_b5`	write dropped on r+w while empty	ASSERTION a_step_up	465 ns

Why b4 slips past every assertion

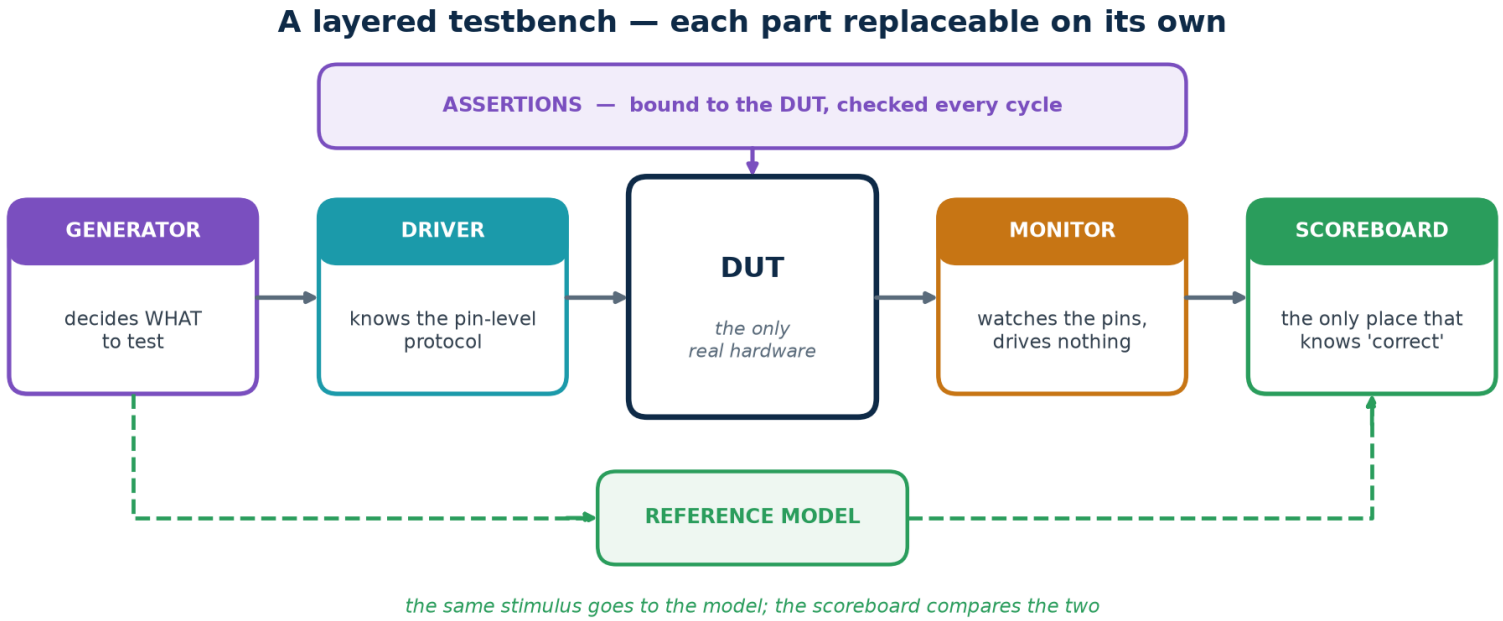
The assertions in sva/fifo_sva.sv describe the CONTROL interface — count, full, empty. fifo_b4 keeps all of those perfectly correct and corrupts the DATA. No property is violated, so nothing fires. The scoreboard, which compares every word, catches it.

Read the fourth row to the class twice

Four bugs are caught by a NAMED assertion, at the exact cycle, with a message that says which rule broke. One is not caught by any assertion — because these assertions describe the CONTROL interface and that bug corrupts DATA. **Assertions catch what you described; scoreboards catch what you compared.** You need both, and neither is a substitute for the other.

A Layered Testbench

A flat testbench works for one design. A layered one survives contact with the second, because the part that decides WHAT to test is separate from the part that knows HOW to wiggle the pins.



Layer	Knows about	Does NOT know about
Generator	the test intent — what scenarios matter	pins, timing, the DUT
Driver	the pin-level protocol and its timing	why this transaction was chosen
Monitor	how to observe the pins	how to drive anything
Scoreboard	what 'correct' means	pins, timing, protocol

The Capstone Testbench

Topic5_Lab/tb/tb_v6_assert.sv — verified: 2027 checks, 895 words in, 895 out

```
// LAYER 3: monitor -- watches the pins and tells the scoreboard what happened.
task automatic mon_apply(bit did_wr, bit did_rd, logic [W-1:0] dat);
  bit was_full  = m_full();          // sample BOTH before applying either
  bit was_empty = m_empty();
  if (did_wr && !was_full)  begin model[mtail % DEPTH] = dat; mtail++; end
  if (did_rd && !was_empty) begin mhead++;                          end
endtask

// LAYER 2: driver -- knows the pin protocol and nothing else.
task automatic drv_cycle(bit do_w, bit do_r, logic [W-1:0] dat, string where);
  @(posedge clk); #1;  wr_en = do_w; rd_en = do_r; wr_data = dat;
  @(posedge clk); #1;  wr_en = 1'b0; rd_en = 1'b0;
  mon_apply(do_w, do_r, dat);
  sb_check(where);
endtask

// LAYER 1: generator -- corners first (cheap, deterministic), then random.
for (int i = 0; i < DEPTH + 2; i++) drv_cycle(1'b1, 1'b0, 8'h10+i[7:0], "fill");
drv_cycle(1'b1, 1'b1, 8'hAA, "read+write while empty");
for (int i = 0; i < cycles; i++) begin
  bit b_w = (($urandom() % 100) < p_wr);      // seeded ONCE, before the loop
  bit b_r = (($urandom() % 100) < p_rd);
  drv_cycle(b_w, b_r, W'($urandom()), "random");
end
```

Directed corners first, then the random body. The corners fail fast and cheaply; the random body finds what the corners did not.

TOPIC 5B · CHECKPOINT

Build It Yourself — Eight Tasks

These are the tasks that make the difference between reading about verification and being able to do it. Full solutions in workbook section T5-B.

Tasks 1–4

1. Add a ninth directed test to V2 that V2 currently lacks. What does it check?
2. Change V3's weights so the FIFO spends most of its time empty. Which coverage bins go MISS?
3. Break the golden FIFO with a one-character change. Which stage catches it — V1, V2 or V3?
4. Write a bug that V3 misses. What kind of bug is it, and what would catch it?

Tasks 5–8

5. Add a coverage bin for "three consecutive idle cycles". Does anything hit it?
6. Add an assertion that would have caught fifo_b4. Why is it harder than the others?
7. Comment out a_step_both and re-run assert.sh. What changes about fifo_b3's diagnosis?
8. Run the same seed twice and diff the transcripts. Then change the seed. Explain both results.

Task 4 is the one that teaches the most

Writing a bug that survives a good testbench forces you to think about what the testbench does NOT check — which is exactly the skill a verification plan is supposed to produce.

TOPIC 5B · MECHANICS

Tasks, Functions and Concurrency in a Testbench

A testbench is not synthesised, so the whole language is available — including everything Topic 4 told you never to use in design code. Delays, while loops, file I/O, hierarchical references: all legal, all useful here.

The testbench-only constructs you will actually use

```
// A TASK may consume time. This is how you package a bus transaction.
task automatic push(input [W-1:0] dat);
  begin
    @(posedge clk); #1;  wr_en = 1'b1; wr_data = dat;
    @(posedge clk); #1;  wr_en = 1'b0;
  end
endtask

// FORK/JOIN runs several stimulus threads at once - a writer and a reader,
// independent of each other, exactly as two masters on a bus would be.
fork
  begin : writer  repeat (100) push($random(seed));      end
  begin : reader  repeat (100) pop();                    end
join

// join_any + disable fork is the standard TIMEOUT idiom in SystemVerilog.
```

automatic matters

`task automatic` gives each call its own copy of the arguments and locals. Without it, two concurrent calls from a fork share one set — and corrupt each other in a way that looks like a DUT bug.

A watchdog is not optional

Every testbench needs a second initial block that prints FAIL and calls $finish after a generous timeout. Without one, a hung DUT stalls the whole regression instead of failing it — and a regression that hangs is a regression nobody runs.

TOPIC 5B · FILE I/O

Golden Vectors — When the Answer Comes From Elsewhere

Sometimes the reference is not a model you write but a file somebody else produced — a C reference implementation, a MATLAB run, the output of the previous chip. The testbench then reads stimulus and expected results from files.

$readmemh, $fopen, $fdisplay, $fclose

```
reg [7:0] stim   [0:1023];
reg [7:0] golden [0:1023];
integer   i, fd, errors;

initial begin
  $readmemh("vectors/stim.hex",   stim);      // one hex value per line
  $readmemh("vectors/golden.hex", golden);
  fd = $fopen("build/mismatch.log", "w");     // and write our own report
  for (i = 0; i < 1024; i = i + 1) begin
    apply(stim[i]);
    if (dut_out !== golden[i]) begin
      $fdisplay(fd, "%0d  sent %h  got %h  expected %h", i, stim[i], dut_out, golden[i]);
      errors = errors + 1;
    end
  end
  $fclose(fd);
end
```

When golden vectors are the right answer

· A published standard with official test vectors (AES, CRC, a codec).
· A bit-exact C or MATLAB reference that already exists and is trusted.
· Regression against the PREVIOUS revision of the same block.

And when they are not

· They fix the stimulus, so they cannot find the corner nobody generated.
· They rot: the file and the specification drift apart and nobody notices.
· A mismatch tells you WHICH vector failed, not which rule was broken.

A Testbench That Survives the Next Project

The testbenches in this lab are parameterised the same way the design is. Change W or DEPTH in one place and everything — stimulus, model, coverage bins — follows. That is not tidiness; it is what makes the testbench reusable at all.

How the clinic runs one testbench against six designs

```
// The DUT is selected by a `define, so ONE testbench runs against SIX designs.
`ifndef DUT
  `define DUT fifo
`endif

`DUT #(.W(W), .DEPTH(DEPTH)) u_dut ( .clk(clk), .rst_n(rst_n), ... );

// and on the command line:
iverilog -DDUT=fifo_b3 -DDUTNAME=\"fifo_b3\" rtl/*.v tb/tb_v3_random.v
xvlog    -d DUT=fifo_b3 ...                    # Vivado
vlog     +define+DUT=fifo_b3 ...               # ModelSim
```

Three habits that make a testbench outlive its design

1. Parameterise everything the DUT parameterises, and derive nothing by hand.
2. Keep the layers separate — a new DUT usually needs a new DRIVER only, and the generator, monitor and scoreboard survive unchanged.
3. Never reach inside the DUT from the scoreboard. A white-box check is fine for debugging and fatal for reuse: the moment the design is refactored, the testbench breaks and nobody can tell whether the design did too.

TOPIC 5B · BY DESIGN TYPE

What to Check, for Each Kind of Block

The six parts of a testbench are always the same. What changes is what you check, and where the bugs hide.

Kind of DUT	The checks that matter	Where the bugs actually are
Combinational	exhaustive if the input space is small; otherwise random + a reference function	boundaries, sign, and width truncation
Registered / pipelined	output correct AND arriving at the right cycle; valid delayed with the data	latency, and control not pipelined with the data
Counter / timer	wrap, load, enable priority, terminal count	off-by-one at the wrap, and at the reload
FSM	every state entered, every arc taken, illegal states recovered from	the arcs nobody drew, and the missing default
Memory / FIFO	order, occupancy, full, empty, simultaneous access	the boundaries: full, empty, and both at once
Protocol / bus	handshake rules as assertions; a checker independent of the DUT	the rule the other end violates, not the one you do
CDC	not by simulation alone — structural CDC checks and a Gray/handshake review	multi-bit buses crossing without a handshake

TOPIC 5B · SYSTEMVERILOG

What SystemVerilog Adds for Verification

Verilog-2005 is enough to build everything in this lab, and doing it by hand once is worth the effort. SystemVerilog then gives you the same things as language features — and you will meet them in every professional environment.

SystemVerilog feature	What it replaces in this lab	Support
logic	the wire / reg decision	everywhere
always_comb / always_ff	always @(*) and the latch you inferred by mistake	everywhere
assert / cover property	hand-written checks scattered through the code	vendor: full; open-source: a subset
covergroup / coverpoint / bins	the twelve integer counters in Lab V4	vendor tools
rand / randc + constraint blocks	the weighted $random in Lab V3	vendor tools
classes, mailboxes, semaphores	the layers written as tasks in Lab V6	vendor tools
interface / modport	the long port lists in every instantiation	vendor tools
queues, dynamic arrays, associative arrays	the fixed model array and its indices	vendor tools

Why this lab is written in Verilog-2005

Because you learn what a covergroup IS by writing twelve counters once, and what a constraint IS by writing the weights yourself. And because it then runs on every tool on every machine, including the ones your students have at home. Move to SystemVerilog as soon as your tool flow allows — but move to it understanding what it is doing for you.

Constraints, Written as Constraints

In Verilog you weight a coin and hope. In SystemVerilog you state the rules and a constraint solver generates values that satisfy them — which is a considerably more powerful thing.

The same generator as Lab V3, expressed declaratively

```
class fifo_txn;
  rand bit        do_wr;
  rand bit        do_rd;
  rand bit [7:0]  data;

  // weights, expressed directly
  constraint c_mix   { do_wr dist { 1 := 55, 0 := 45 };
                       do_rd dist { 1 := 45, 0 := 55 }; }

  // and rules the generator must respect
  constraint c_burst { (do_wr && do_rd) -> data inside {[8'h80:8'hFF]}; }
endclass

fifo_txn t = new();
repeat (2000) begin
  if (!t.randomize()) $fatal(1, "constraints are unsatisfiable");
  drv_cycle(t.do_wr, t.do_rd, t.data);
end
```

What the solver buys you

· Constraints compose: add one and the others still hold.

· **`randomize() with { ... }`** adds a temporary constraint at one call site — a directed test written as a narrowing of the random one.

And the trap

An over-constrained set has no solution and **`randomize()`** returns 0. If you ignore the return value the test silently drives the same unrandomised value for ever — and passes. Always check it, and always $fatal on failure.

Covergroups — the Same Twelve Bins, Declared

Lab V4's coverage model, as a SystemVerilog covergroup

```
covergroup cg_fifo @(posedge clk);
  cp_op : coverpoint {wr_en, rd_en} {
            bins idle  = {2'b00};
            bins wr    = {2'b10};
            bins rd    = {2'b01};
            bins both  = {2'b11};
          }
  cp_occ: coverpoint count {
            bins empty = {0};
            bins mid   = {[1:DEPTH-1]};
            bins full  = {DEPTH};
          }
  // the CROSS is where the real value is: read+write WHILE empty, and
  // read+write WHILE full, are cells of this cross - and both were bugs.
  x_op_occ: cross cp_op, cp_occ;
endgroup

cg_fifo cg = new();          // sampled automatically on every posedge clk
```

The cross is the point

cp_op has 4 bins and cp_occ has 3, so the cross has 12 cells — and two of them, "both while empty" and "both while full", are exactly where two of the five planted bugs live. **A cross says "this situation, in that state" in one line** — the combination hand-written counters make tedious, and that people skip.

Coverage is reported and merged by the tool

`vsim -coverage ; coverage report -details` and merged across a regression with **`vcover merge`**. That is the vendor equivalent of the awk merge in scripts/coverage.sh.

What UVM Is, and When It Is Worth It

The Universal Verification Methodology is a SystemVerilog class library that standardises exactly the layers in Lab V6. It is what most professional verification environments are built from — and it is considerable overhead for a small block.

Lab V6 calls it	UVM calls it	What UVM adds
generator	uvm_sequence / uvm_sequencer	reusable, layered, randomisable sequences
driver	uvm_driver	a standard handshake with the sequencer
monitor	uvm_monitor	publishes transactions to any number of subscribers
scoreboard	uvm_scoreboard	standard comparison and reporting
(the whole thing)	uvm_env / uvm_agent	a package you can instantiate twice, or reuse on the next project
$display	uvm_info / uvm_error with verbosity	filterable, gradable reporting
(nothing)	the factory and config_db	swap a component or a setting without editing the environment

Worth it when

· The block has a real protocol interface that other blocks also use.
· Several people work on the environment at once.
· The environment will be reused across projects, or at chip level.

Not worth it when

· The block is a FIFO, and Lab V6 already verifies it in 160 lines.
· Nobody on the team has used UVM before and the schedule is four weeks.
· Your simulator licence does not include the class library.

TOPIC 5B · FORMAL

Formal Verification — Proof Instead of Sampling

Simulation samples the state space. Formal tools SEARCH it. Given the same assertions you already wrote, a formal engine tries to prove no input sequence can ever violate them — or produces a counter-example waveform that does.

WHAT YOU GET	For a property that is PROVEN, no amount of simulation adds anything: it holds for every input sequence, for ever. For one that fails, you get the shortest counter-example — usually far easier to understand than a random failure at cycle 40 000.
WHAT IT COSTS	The state space can explode. Formal works beautifully on control logic — FIFOs, arbiters, protocol adapters, CDC handshakes — and poorly on wide datapaths such as multipliers, where the engine runs out of memory rather than returning an answer.
THE OVERLAP	The assertions in sva/fifo_sva.sv are already formal properties. That is the practical point: writing assertions for simulation costs you nothing extra and gives you a formal testbench for free if you later get access to a tool.
WHERE IT IS USED	Almost universally for connectivity checks, register maps, clock-domain crossings, arbiters and cache coherence — and almost never as a replacement for simulation at the system level.

TOPIC 5B · CDC

Why Simulation Alone Cannot Verify a Clock Crossing

This is the most important limitation to state out loud. A simulator has no model of metastability. It resolves every flip-flop to a clean 0 or 1, so a design with a genuinely broken clock crossing simulates perfectly and fails intermittently in the lab.

What simulation WILL catch

· Functional errors in the handshake protocol itself.

· Data being sampled before it was stable, if you model the delay.

· A Gray-code pointer that is not actually Gray.

· With randomised clock phases, some — not all — sampling races.

What it will NOT catch

· Metastability. There is no such value in the simulator.

· A missing synchroniser: two flip-flops or none simulate identically.

· A multi-bit bus crossing without a handshake — in simulation all the bits arrive together, which is exactly what does not happen on silicon.

So CDC is verified structurally, not dynamically

Every professional flow runs a dedicated CDC checker that reads the netlist, identifies every path between unrelated clocks, and demands a recognised synchroniser structure on each one. Review the crossings by eye as well: **two flip-flops for a single-bit level, Gray coding or a handshake for a bus, and never bit-by-bit synchronisation of a vector.**

TOPIC 5B · METRICS

How a Project Decides Verification Is Finished

"We ran all the tests and they passed" is not a sign-off criterion. These are — and every one of them is a number somebody outside the team can read.

Metric	What it means	Typical sign-off bar
Functional coverage	spec-level situations reached	100% of the plan, or every hole waived in writing
Code coverage	lines, branches, conditions, toggles exercised	95–100%, with the rest explained
Assertion coverage	each assertion actually evaluated, not vacuously true	every assertion hit at least once
Regression pass rate	seeds passing over the last N runs	100%, sustained, not 'usually'
Bug discovery rate	new bugs found per week	flattened, and staying flat
Bug severity trend	are the new ones getting less serious?	no high-severity bugs for N weeks

The bug-rate curve is the one experienced teams watch

If you are still finding serious bugs at the same rate as last month, you are not near the end however good the coverage number looks. If the rate has flattened AND coverage is closed AND the regression is green, you have as much evidence as this method can give you.

TOPIC 5B · CHECKPOINT

Six Questions on the Advanced Material

Discussion questions — there is no code to type. Answers in workbook section T5-B.

Questions 1–3

1. Your testbench uses golden vectors from a trusted C model and every vector passes. Name two classes of bug this cannot find.

2. Why does a scoreboard that reads the DUT's internal pointers make the testbench worse, even though it makes debugging easier?

3. A cross of a 4-bin and a 3-bin coverpoint has 12 cells, and two of them are impossible by construction. What do you do about them?

Questions 4–6

4. randomize() returns 0 and the test still passes. Explain how, and what you should have written.

5. A design with no synchroniser at all simulates perfectly. Why, and what actually catches it?

6. Coverage is 100% and the regression is green, but the team is still finding two serious bugs a week. Are you finished? What does that tell you?

Question 6 is the one to spend time on

It is the difference between measuring what you tested and knowing whether the measurement was the right one. A closed coverage model that misses a whole category of behaviour reports 100% and means nothing — which is why the verification plan is reviewed by somebody other than its author.

SUBTOPIC 5C

Simulation and Debugging of RTL Designs

How the simulator actually works, and what to do when the answer is wrong.

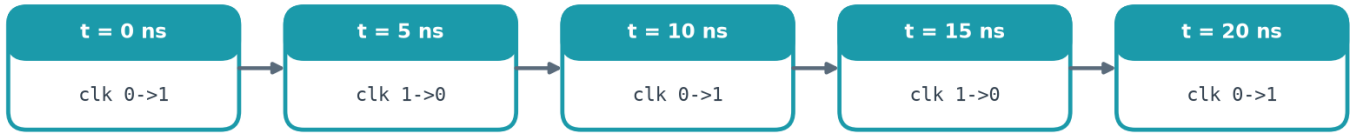
- The event-driven engine, and what happens inside one time step
- Analyse, elaborate, run — the same three steps in every tool
- Waveforms: dumping, reading, and saving a view
- A debugging procedure: bisect in time, bisect in space, chase the x
- Symptom-to-cause, and how to make simulation fast enough to live with

A Simulator Runs Events, Not Instructions

A digital simulator does not step through your code. It keeps a queue of scheduled EVENTS ordered by time, and at each time it wakes only the blocks sensitive to what changed. Understanding that explains almost every surprising thing a simulation does.

How a simulator actually works — events, not instructions

A digital simulator does not step through your code. It keeps a queue of scheduled EVENTS, ordered by time.



At each time the simulator wakes only the blocks SENSITIVE to what changed, runs them to completion, and moves on.

Why this is fast

Nothing is evaluated unless one of its inputs changed. A design that is mostly idle costs almost nothing to simulate — which is why a clock that never stops is expensive.

Why time can stand still

Several events can be scheduled at the SAME simulation time. The simulator works through them all before the clock advances. That is where races come from.

Why simulation time is not wall-clock time

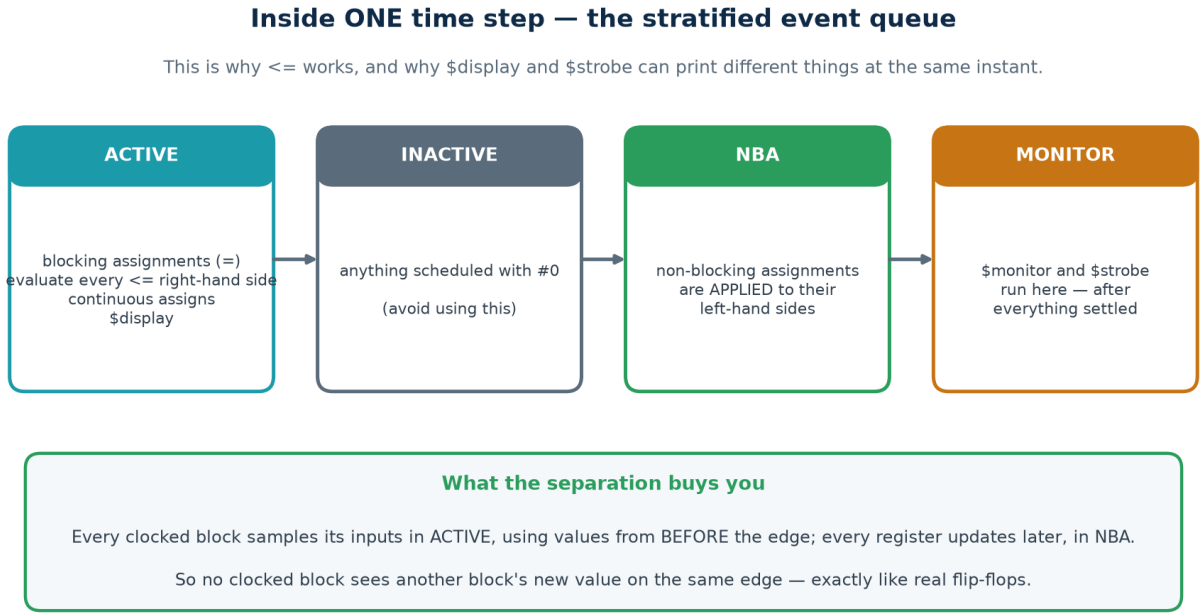
A million simulated clock cycles may take a second or an hour, depending entirely on how much of the design is switching. Time in the simulator advances only when there is nothing left to do at the current time.

Why a loop with no delay hangs

`while (!done) ;` in a testbench never lets simulation time advance, so done can never change. The simulator is not stuck — it is doing exactly what you wrote. Every wait loop needs an @ or a #.

The Stratified Event Queue

Several events can be scheduled at the SAME simulation time. The standard defines regions that are processed in order, and that ordering is what makes non-blocking assignment model a flip-flop correctly.



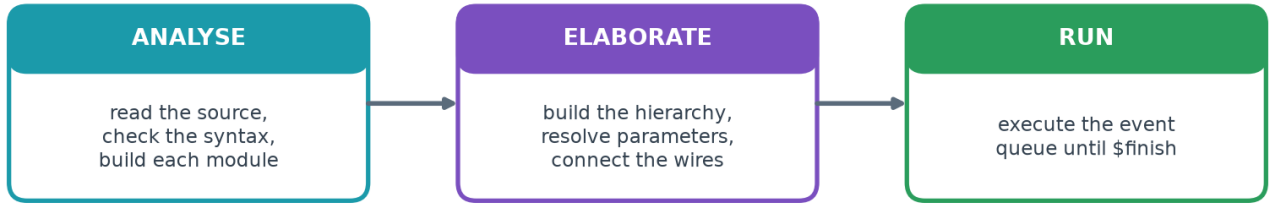
The practical consequence you will meet first

`$display` runs in the ACTIVE region, so at a clock edge it prints the value from BEFORE the edge. **`$strobe`** runs in the MONITOR region, after every non-blocking update has landed, so it prints the settled value. If a printout disagrees with the waveform by exactly one cycle, this is why.

Analyse, Elaborate, Run

Every simulator you will ever use does the same three things. Only the command names change — which is why moving between Icarus, xsim and ModelSim is a morning's work rather than a new skill.

Every simulator does the same three steps



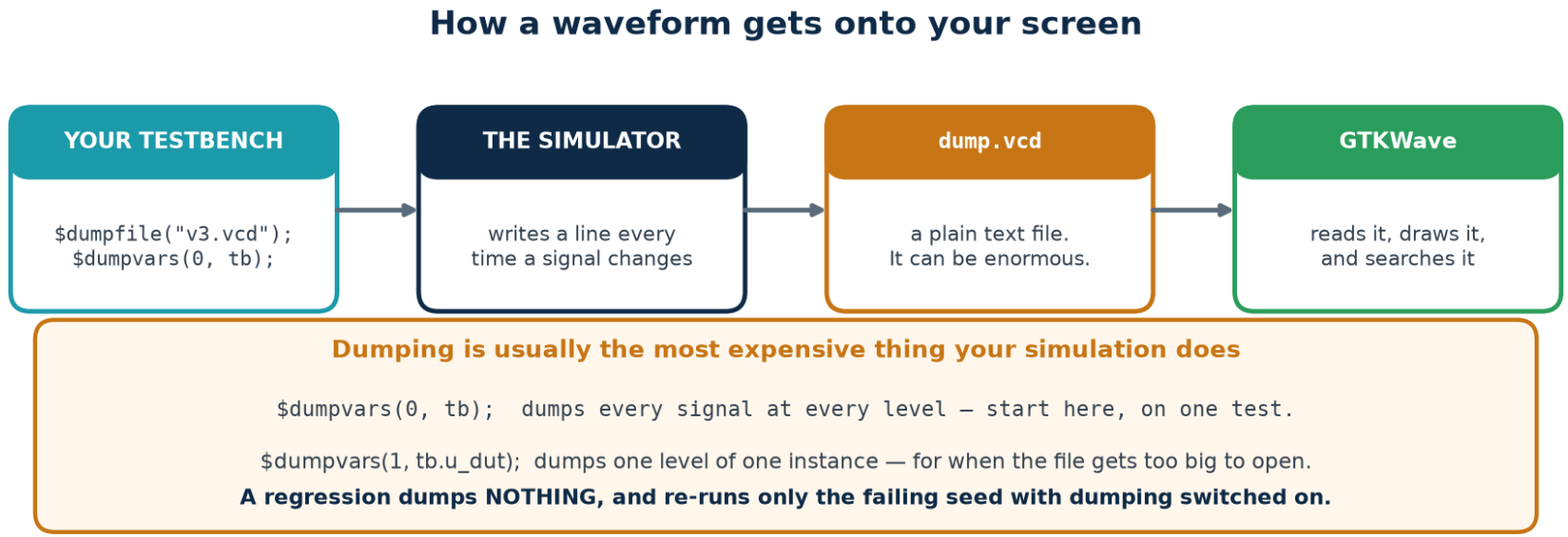
	analyse	elaborate	run
Icarus Verilog	iverilog -o sim.vvp ...	(part of iverilog)	vvp sim.vvp
Verilator	verilator --binary ...	(part of the build)	./obj_dir/<top>
Vivado xsim	xvlog <files>	xelab -debug typical <top>	xsim <snap> -runall
ModelSim / Questa	vlog <files>	(part of vsim)	vsim -c work.<top>; run -all

Where each kind of error appears

ANALYSE reports syntax errors and undeclared names — file and line. **ELABORATE** reports missing modules, port mismatches and parameter problems. **RUN** is where your logic is finally wrong. Knowing which step failed tells you which kind of mistake you made.

How a Waveform Gets Onto Your Screen

The simulator does not draw waveforms. Your testbench asks it to write a change-log file, and a separate program draws that. Knowing this explains why dumping is optional, why it is slow, and why the file can be enormous.

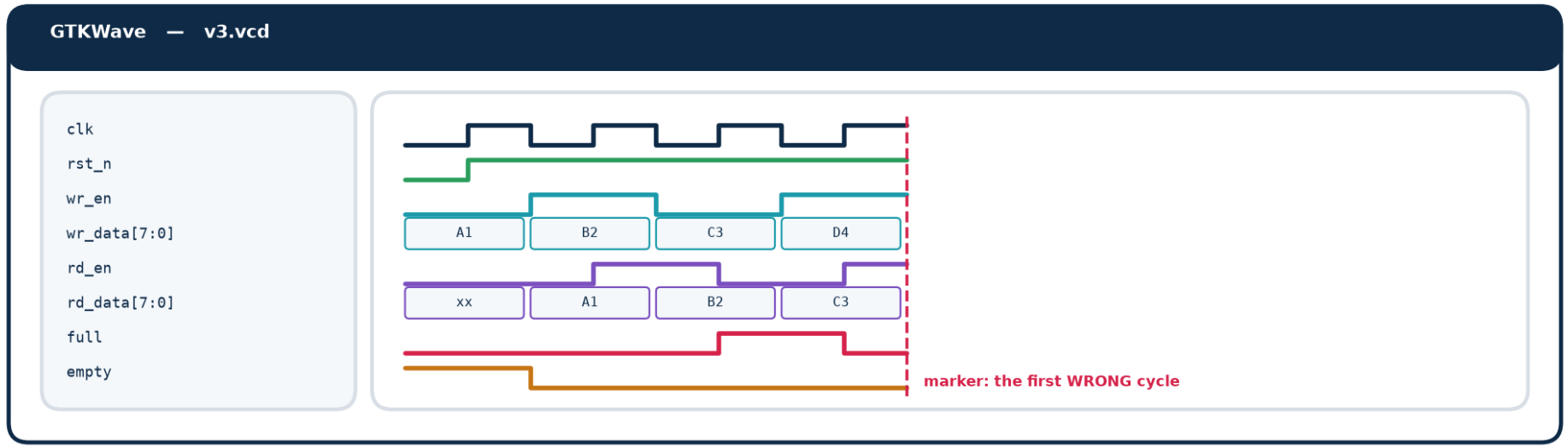


Format	Written by	Note
VCD	any simulator; $dumpvars	plain text, universal, large
FST	Icarus (-fst), GTKWave	compressed VCD; much smaller and faster
WLF	ModelSim / Questa	native, fast, read by their own viewer
FSDB	commercial (Verdi)	the industry standard on large projects

Reading a Waveform Viewer Properly

Most students open a viewer, drag in every signal, and stare. Four habits turn it from a wall of lines into an instrument.

Reading a waveform viewer



Group and order

Stimulus at the top, DUT outputs next, internals last. A jumbled list hides the answer in plain sight.

Set the radix

A counter in binary is unreadable; in hex the pattern is obvious. Right-click the signal to change it.

Use the marker

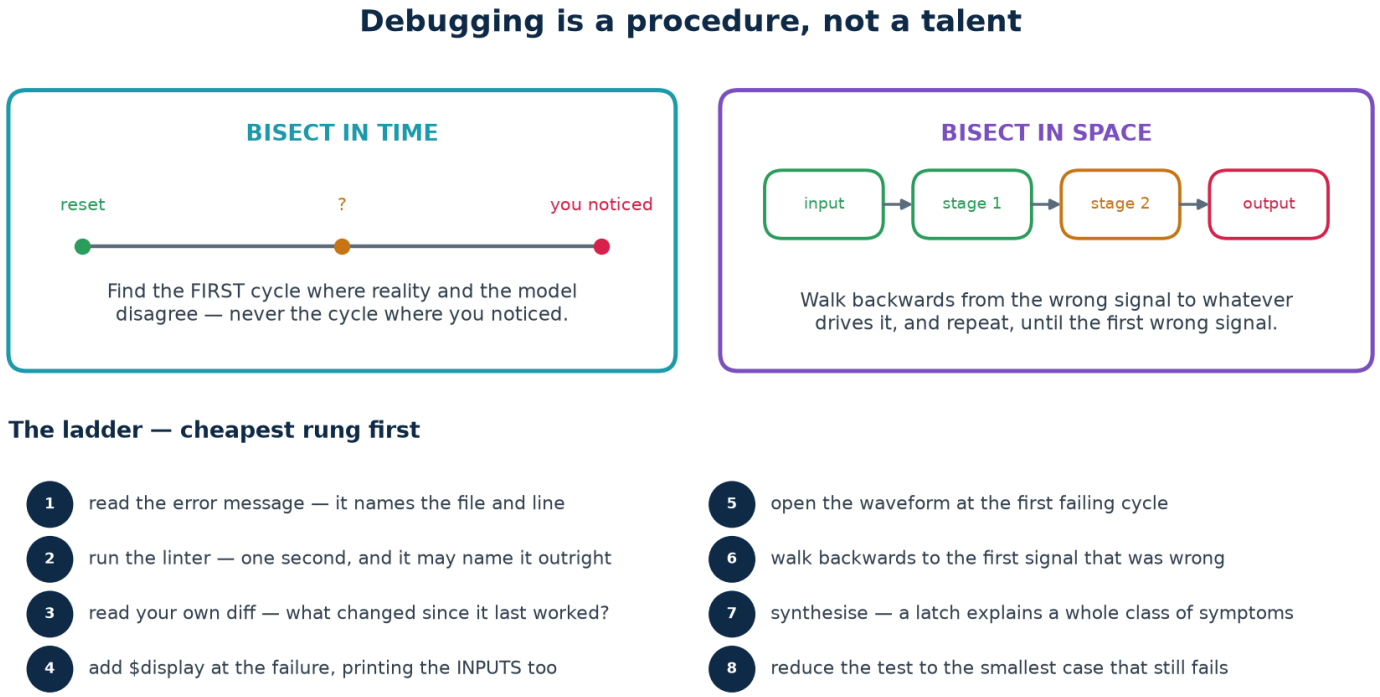
Put it on the FIRST wrong cycle named in the transcript, then walk backwards, signal by signal.

Save the view

File → Write Save File writes a .gtkw. Commit it, open with `gtkwave v3.vcd wave/v3_fifo.gtkw` and everyone sees the SAME picture.

A Procedure, Not a Talent

Fast debuggers are not more clever — they are more systematic. Two searches, and a ladder of checks run in cost order.



The single most important habit

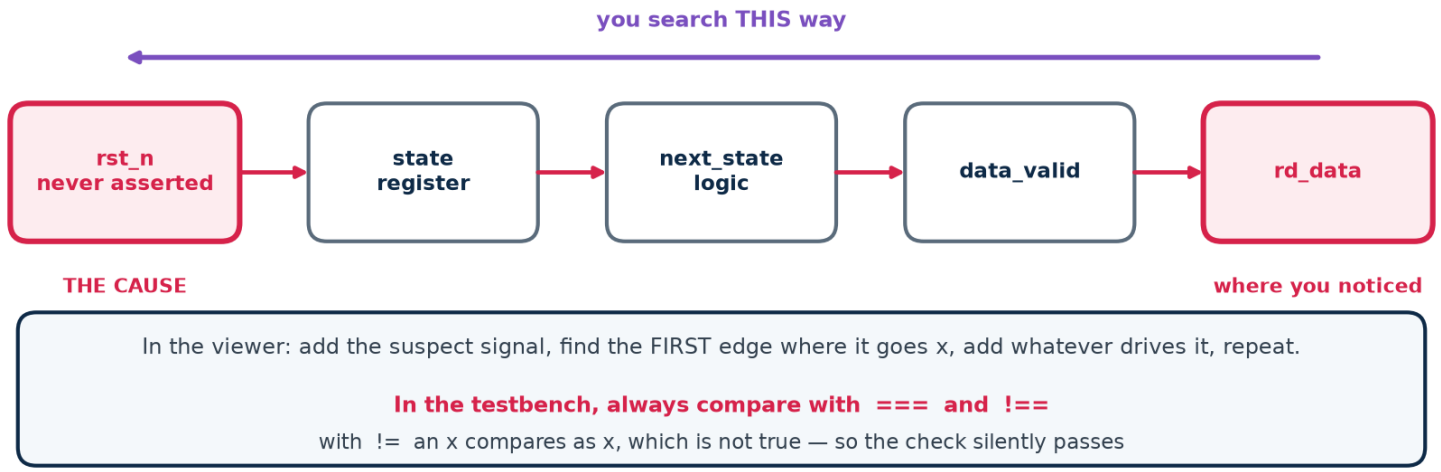
Find the FIRST moment things went wrong, never the moment you noticed. A wrong output is usually many cycles downstream of the cause, and every cycle you spend looking at the output is a cycle not spent looking at the cause.

Chasing an x

An x is the simulator saying "I cannot determine this". It spreads through every expression it touches, so by the time you see it on an output it may have travelled through ten signals.

Chasing an x back to its source

x is contagious. The signal where you NOTICED it is almost never the signal that caused it.



In the viewer: add the suspect signal, find the FIRST edge where it goes x, add whatever drives it, repeat.

In the testbench, always compare with `===` and `!==`

with `!=` an x compares as x, which is not true — so the check silently passes

The five usual causes

1. A register that was never reset. 2. A wire nothing drives — often a typo in a port connection. 3. A wire TWO things drive.

4. Reading past the end of a vector or an array. 5. Arithmetic on a value that was already x.

And one prevention

`` `default_nettype none `` at the top of every file turns a misspelt port connection from a silent undriven wire into a compile error naming the line. It costs one line and removes an entire category of x.

Symptom to Cause

Most failures have a signature. Recognising it is most of the debugging — and this table is worth printing and putting on the wall of the lab.

Symptom to cause — the table to put on the lab wall

What you see	Almost always means	Where to look first
x on everything, from time 0	a register was never reset	the reset path
x appears part-way through	two drivers, or a read past the end of a vector	every assign to that signal
off by one cycle	a check sampled before the NBA update landed	the testbench, not the DUT
off by one COUNT	a boundary: <= where < was meant	the comparison in the RTL
a pulse is half a cycle wide	stimulus driven on the clock edge	the testbench driver
works for some data, not others	width truncation somewhere	run the linter
passes at one parameter, fails at another	a constant sliced too narrow	every [W-1:0] on a constant
testbench passes, hardware fails	casex, full_case, or an initial block	grep the RTL for all three
FSM stuck in a state it cannot leave	no default branch	the next-state case
simulation is unbearably slow	waveform dumping, or a testbench loop with no delay	$dumpvars scope

Every row is a real failure mode from Topic 4 and Topic 5. Recognising the signature is most of the debugging.

TOPIC 5C · SPEED

Making Simulation Fast Enough to Live With

A regression that takes all night gets run once a week; a regression that takes five minutes gets run on every commit.
Simulation speed is therefore a verification-quality issue, not a convenience.

What costs time	Why	What to do about it
Waveform dumping	every signal change becomes a line of text	dump nothing in a regression; re-run the failing seed with dumping on
Dumping the whole hierarchy	$dumpvars(0, tb) includes every instance	$dumpvars(1, tb.u_dut) once you know where to look
A clock that never stops	every edge wakes every clocked block	stop the clock when the test is idle, if the design allows it
Very fine timescale	1ps precision means more scheduling work	1ns/1ps is plenty for RTL; you are not doing SPICE
$display in a hot loop	formatting and I/O are slow	print on events, not on cycles
Long random runs at one seed	one long run is not better than many short ones	many shorter runs, different seeds — and they parallelise

Verilator compiles the design to C++ and is often 10–100× faster than an interpreted simulator — which is why it is used for long random regressions even on projects whose sign-off simulator is a vendor tool.

Software Tools — Installation and Flow

The syllabus specifies Vivado Design Suite and ModelSim. The open-source chain does the same jobs and installs in a minute.

- What each tool does, and which one to reach for
- Installing the open-source verification flow
- Driving Vivado xsim and ModelSim / Questa
- Regression scripting, and what a CI loop looks like

The Verification Jobs, and the Tool for Each

Four jobs — lint, simulate, view, measure. Every flow does all four; only the command names change.

The verification jobs, and the tool that does each one

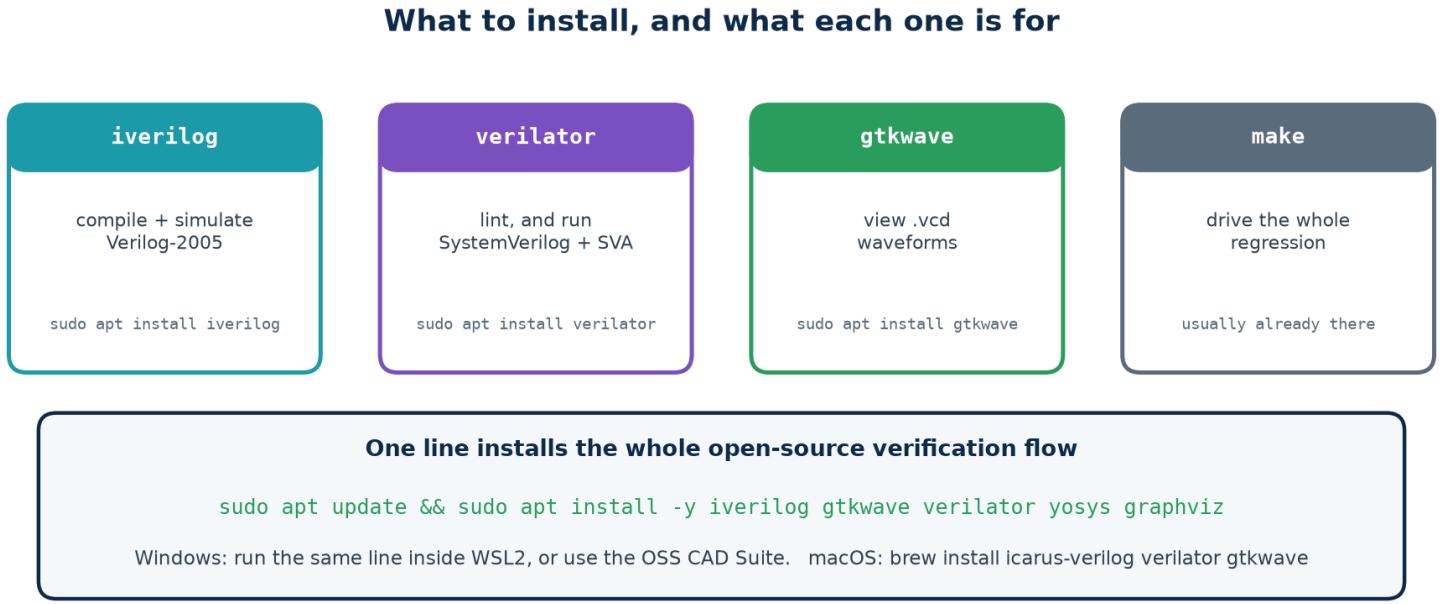
job	open-source	Vivado	ModelSim / Questa
static checks (lint)	verilator --lint-only -Wall	report_methodology	vlog warnings
simulate Verilog	iverilog + vvp	xvlog / xelab / xsim	vlog / vsim
simulate SystemVerilog	verilator --binary --timing	xvlog -sv	vlog -sv
concurrent assertions	verilator --assert (subset)	full SVA	full SVA
waveforms	GTKWave (.vcd)	built-in wave window	wave window (.wlf)
code coverage	verilator --coverage	-cover, report_coverage	vlog -cover bcesx
functional coverage	by hand (see Lab V4)	SV covergroups	SV covergroups
regression	shell / Makefile	TCL batch	TCL .do batch

Everything in Topic5_Lab was run with the open-source column. The vendor scripts are working templates, not captured runs.

Honesty note for the trainer

Every result quoted in this deck was produced by the open-source column — Icarus Verilog 12.0 and Verilator 5.020 — on the code in Topic5_Lab. The Vivado and ModelSim scripts in the lab are working templates written from standard, version-stable commands, but they were NOT executed, because neither tool is installed in the authoring environment. Check them against your release before the session.

The Open-Source Verification Flow



Copy-paste installation

```
# ---- Ubuntu / Debian / WSL2 -------------------------------------------
sudo apt update && sudo apt install -y iverilog gtkwave verilator yosys graphviz

# ---- verify -----------------------------------------------------------
iverilog -V | head -1     # Icarus Verilog version 12.0
verilator --version       # Verilator 5.020
gtkwave --version
```

macOS: `brew install icarus-verilog verilator gtkwave` Windows: run the Ubuntu line inside WSL2 (`wsl --install` in an admin PowerShell), or unzip the OSS CAD Suite.

Vivado xsim and ModelSim / Questa

Vivado simulator (xsim)

```
xvlog -d DUT=fifo_b1 rtl/*.v tb/tb_v3_random.v
xelab -debug typical tb_v3_random -s sim
xsim sim -runall -testplusarg SEED=1
```

· **`-d NAME=value`** passes a `define, exactly like -D to iverilog.

· **`-testplusarg`** passes a plusarg, so the SAME seed gives the same run as under Icarus.

· Supports the FULL SystemVerilog assertion language, including the ranged delay forms the open-source flow cannot handle.

ModelSim / Questa

```
vlib work
vlog -sv +define+DUT=fifo_b1 rtl/*.v sva/*.sv tb/*.sv
vsim -voptargs=+acc -assertdebug +SEED=7 work.tb_v6_assert
assertion fail -action break -r /*
run -all
```

· **`+acc`** keeps signal visibility — without it the optimiser removes the very signals you want to see.

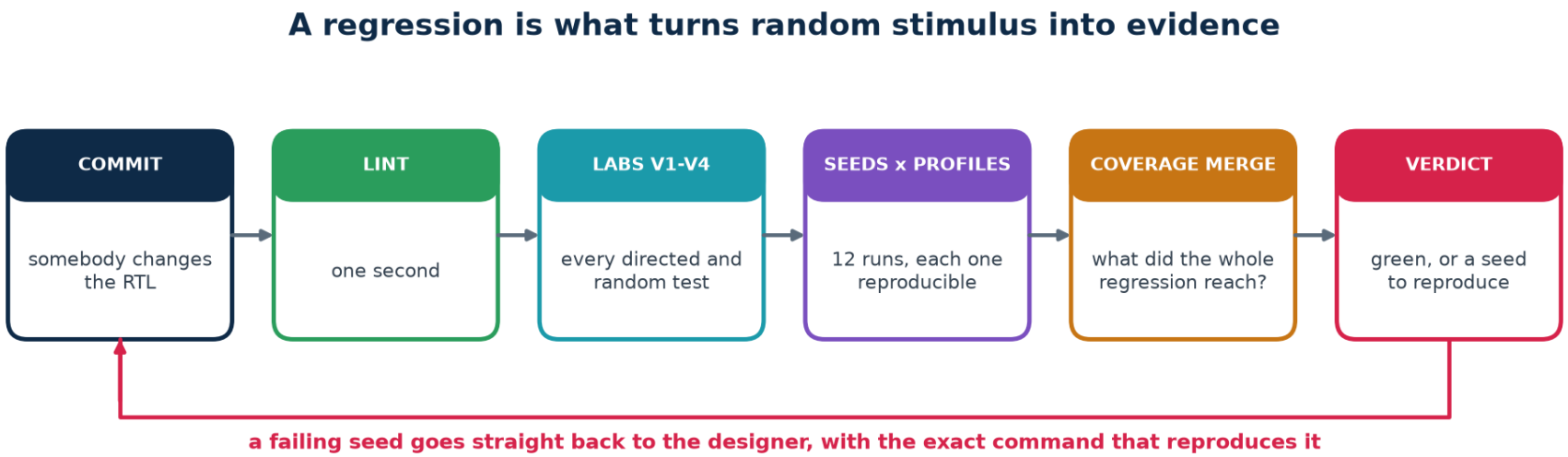
· **`-assertdebug`** reports every assertion pass and failure, and can break on the first failure.

Both flows are scripted in the lab

`scripts/vivado_sim.tcl` takes a lab and a DUT as arguments; **`scripts/modelsim_run.do`** takes them as -g parameters. Both cover all six labs and all six designs, so the clinic can be reproduced on the vendor tools exactly as it is on the open-source ones.

What Turns Random Stimulus Into Evidence

A single random run is one sample. A regression — many seeds, several profiles, run automatically after every change — is what makes it evidence.



Run it on every commit. A regression that is run "when we remember" is a regression that has already stopped working.

```
$ ./scripts/regress.sh 4
  regression on fifo : 4 seeds x 3 profiles
  ...
  12 passed, 0 failed
  REGRESSION CLEAN
```

When a run fails, the script prints the exact command that reproduces it. That one line is the difference between a bug report somebody can act on and one they cannot.

TOPIC 5 · PRACTICAL

The Lab Programme

Six labs, thirty-four hours, inside the syllabus practical component: simulating and verifying the functionality of RTL designs.

- V1 naive · V2 model + corners · V3 random · V4 coverage · V5 clinic · V6 layered + assertions
- One DUT, five planted bugs, and a measured catch rate for every stage
- Scripts for the open-source chain, Vivado and ModelSim
- Assessment rubric and the troubleshooting table

TOPIC 5 · LABS

The Six Labs

Every lab builds on the one before, against the same device under test. Nothing is thrown away; the testbench grows.

Lab	h	What you build	What it teaches
V1 naive	3	a directed testbench with all six parts	structure, a verdict, and why passing is not proving
V2 model	6	a reference model and the boundary cases	expected values come from the spec, not from you
V3 random	6	weighted constrained-random, seeded	reproducibility, and the corner nobody writes
V4 coverage	6	a coverage model, sampled and merged	what did I actually test, and when do I stop
V5 clinic	5	diagnosis of five unknown bugs	bisection, x-chasing, reading a waveform properly
V6 capstone	8	a layered environment with assertions	separation of concerns; assertions vs scoreboards

Thirty-four hours, from the syllabus practical bullet "simulating and verifying the functionality of RTL designs".

What every lab produces

· A lint run that is clean.
· A simulation transcript ending in a machine-readable PASS or FAIL.
· From V4 on, a coverage report with a closure verdict.
· One sentence from the student on what their testbench would NOT catch.

Running short of time?

Do not cut V5, the clinic. It is the only exercise where students meet a bug they did not plant, with no hint about what it is — which is the actual job. Shorten V1 instead: the naive testbench can be read rather than typed, provided the room still watches it pass on all five broken designs.

Worth ten minutes of the session

Have every student plant ONE bug in a copy of the golden FIFO and hand it to a neighbour, whose testbench must catch it and name it.

How to Run the Session

OPEN WITH THE CLINIC	Do not begin with theory. Run ./scripts/clinic.sh live, in front of the room, before explaining anything. The table — 0 of 5, 4 of 5, 5 of 5 — does the persuading, and everything afterwards is the explanation of why.
READ THE SPEC FIRST	Have the class read the header comment of rtl/fifo.v and list, on the board, every rule it states. That list IS the verification plan. Only then look at a testbench.
BUILD, DO NOT COPY	V1 and V2 are typed from scratch, not copied. The supplied versions are for comparing against afterwards. The point is the decisions, and you only meet them by making them.
BREAK IT ON PURPOSE	Every student plants one bug in a copy of the golden FIFO and hands it to a neighbour, whose testbench must catch it. This single exercise teaches more about verification than any lecture, and the room enjoys it.
CLOSE WITH COVERAGE	Finish on ./scripts/coverage.sh. Passing tests feel like the end; the coverage report showing three MISSes is what teaches that they are not.

What Your Students Will Hit

Testbench problems, not design problems. Nine out of ten support questions in this topic are one of these.

Symptom	Almost always means	Fix
Everything is x from time 0	reset never asserted, or the DUT port is misspelt	check reset polarity; `default_nettype none
The check passes when the value is x	== used instead of ===	use === and !== in every testbench check
The design looks one cycle late	sampled before the NBA update landed	sample after the NEXT edge, or use $strobe
A pulse is half a cycle wide	stimulus driven ON the clock edge	drive at #1 after the edge, never on it
Random test passes but drove 9 transactions	$urandom(seed) called in the loop	seed once with void'($urandom(s)), then $urandom()
The correct design fails	the reference model is wrong	sample full/empty once, before applying either operation
Simulation never ends	no $finish, or a wait loop with no delay	add $finish and a watchdog timeout
The waveform has no signals	$dumpvars scope too narrow, or a stale VCD	$dumpvars(0, tb); and re-run before re-opening
Coverage is 100% but bugs escape	the coverage model is too coarse	add bins for SEQUENCES, not just states

What 'Done' Looks Like

Assess the testbench, not the design. Give every student the same correct FIFO and the same five broken ones, and grade them on what their own testbench catches.

Level	Evidence
Pass	The testbench is self-checking, prints PASS or FAIL, terminates, and catches at least three of the five planted bugs.
Good	All of the above, plus: it catches all five, and the student can say which check caught each one and why.
Strong	All of the above, plus: a coverage model with a closure verdict, and a regression over several seeds and profiles.
Excellent	All of the above, plus: assertions bound to the DUT, and the student can explain a bug that assertions cannot catch and why the scoreboard is needed too.

One question to ask every student

"Show me a change to the design that your testbench would NOT catch." If they can answer immediately, they understand verification. If they cannot, they have been testing rather than verifying.

Terms Used in This Topic

Term	Meaning
Verification	establishing that a design does what its specification says.
Validation	establishing that the specification was the right one. A different question, usually answered by somebody else.
DUT	device under test — the design the testbench instantiates.
Testbench	a port-less module that instantiates the DUT, drives it, and decides automatically whether the results were correct.
Self-checking	the testbench forms its own verdict; no human reads a waveform to decide.
Reference model	an independent implementation of the specification, written from the spec, used to compute expected results.
Scoreboard	the component that compares DUT behaviour against the model.
Directed test	a test whose case and expected result you wrote by hand.
Constrained-random	stimulus generated by weighted random choice within constraints you specify.
Seed	the number that makes a random run reproducible.
Regression	the full set of tests, run automatically after every change.
Code coverage	which lines, branches, conditions and toggles the tests reached.
Functional coverage	which SPECIFICATION-level situations the tests reached. Written by you.
Coverage hole	a bin nothing ever hit — a gap in testing, not a failure.
Assertion	a rule stated in the design's own language and checked on every clock edge.
Cover property	the same syntax used to RECORD that a situation occurred, rather than to require it.
VCD / FST / WLF	waveform file formats — plain, compressed, and vendor-native.

TOPIC 5 · RECAP

The Ten Things That Matter

- A testbench is finished when it would FAIL if the design were wrong — not when it passes.
- Every testbench has six parts, and the one people omit is the verdict and the watchdog.
- Drive stimulus just AFTER the clock edge; sample just before the next one. Never on the edge.
- Expected values come from an independent model of the SPECIFICATION, never from the RTL.
- A model must sample the state ONCE, before applying any operation — exactly as the hardware does.
- Constrained-random finds the corner nobody writes a directed test for. Seed it, and print the seed.
- Coverage is what turns "it passed" into "here is what was tested". Merge it across the regression.
- Assertions catch a broken rule at the cycle it breaks; scoreboards catch what no assertion describes.
- Debug by procedure: first wrong cycle, then walk backwards. Never start at the output.
- Lint, then simulate, then measure. Run the regression on every commit, or it will stop working.

Where this leads

Topic 6 takes these verified designs into timing — constraints, setup and hold analysis, and closure. The verification skills here do not stop being needed: every timing fix is a design change, and every design change has to be re-verified. That is what the regression is for.

RTL Simulation and Verification

Deck · workbook · one DUT · five planted bugs · six testbenches · three toolchains.

- Slides: this deck, for delivery
- Workbook: Module2_Topic5_Tutorial_Practice_Workbook.docx — tutorials, exercises and full solutions
- Lab: Topic5_Lab/ — rtl/, tb/, sva/, scripts/, all verified end to end
- Next: Module 2 Topic 6 — timing constraints and analysis